

Digital Logic Design

Semester II
FY

Course Code
316U06C203
Jan – May 2026

Combinational Circuits

Contents

- **Half and Full Adder,**
- **Half and Full Subtractor,**
- **Four Bit Binary Adder,**
- **Four Bit Binary Subtractor (1's and 2's complement method)**
- **Controller Adder / Subtractor**
- **Multiplexers and multiplexer trees,**
- **Decoders**
- **Encoders**
- **One bit, Two-bit ,4-bit Magnitude Comparator**

Half Adder

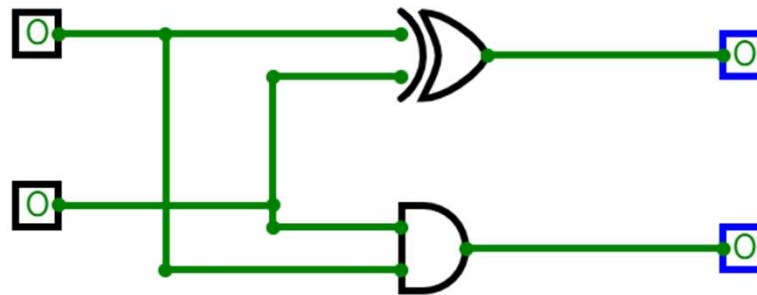
Half adder Truth Table

A	B	Sum	Cy
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half adder

$$\text{Sum} = \Sigma m(1,2) = A \text{ xor } B$$

$$\text{Carry} = \Sigma m(3) = A \text{ and } B$$

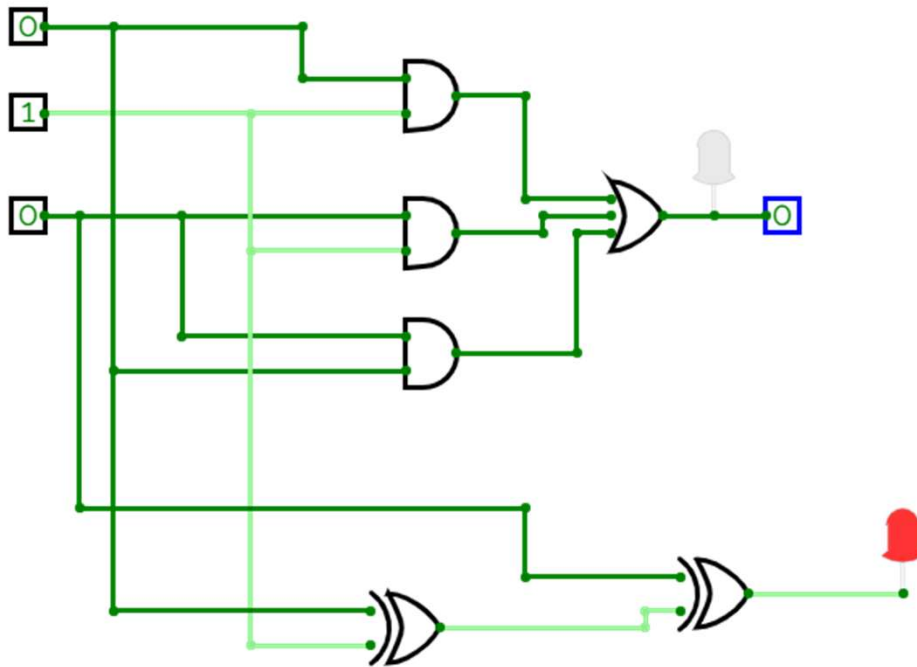


Full Adders

Full adder

$$\begin{aligned}\text{Carry} &= \Sigma m(3,5,6,7) \\ &= A.B + A.C_{in} + B.C_{in}\end{aligned}$$

$$\begin{aligned}\text{Sum} &= \Sigma m(1,2,4,7) \\ &= A \text{ xor } B \text{ xor } C_{in}\end{aligned}$$



Full adder Truth Table

A	B	Cin	Sum	Cy
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Adder using 2 half Adders

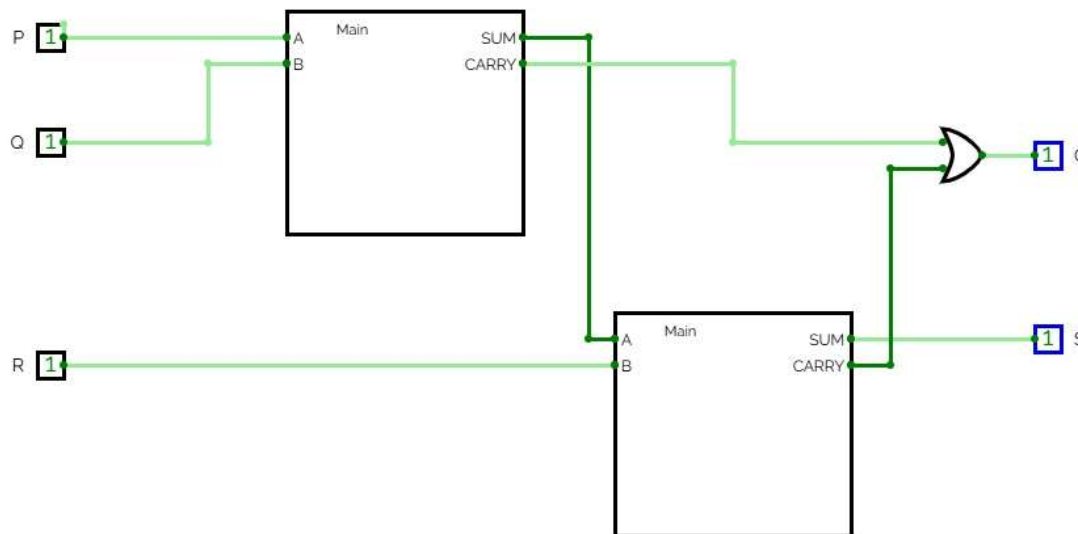
Half Adder: Sum = $A \text{ xor } B$, Carry = $A \cdot B$

Full Adder: Sum = $A \text{ xor } B \text{ xor } C_{in}$, Carry = $AB + BC_{in} + A C_{in}$

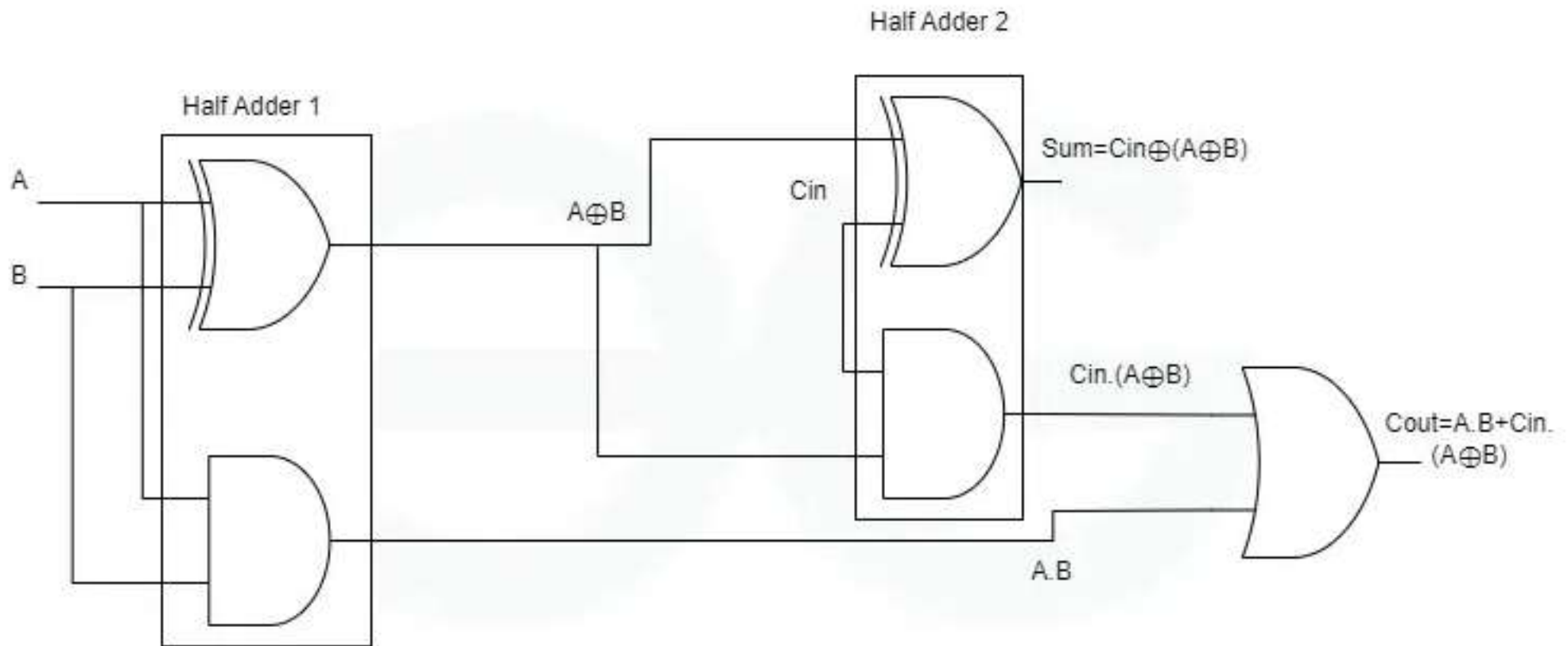
Full adder using 2 half adders

Carry = $A.B + (A \text{ xor } B) . C_{in}$

Prove that it is same as $AB + BC_{in} + A C_{in}$

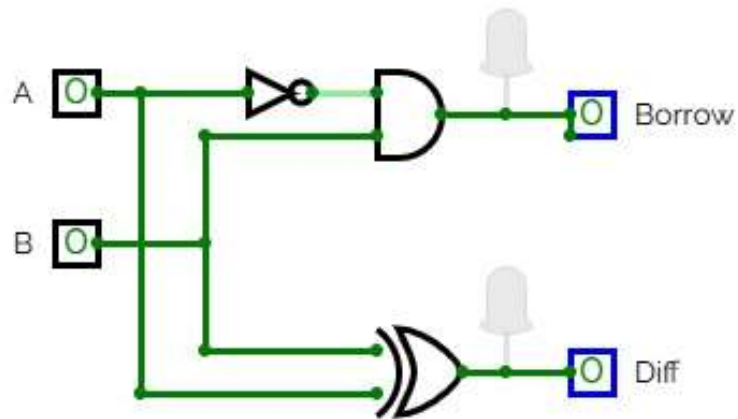


Full Adder using 2 half Adders



Half Subtractor

$$\text{Diff} = A \text{ XOR } B, \text{Bo} = A' \cdot B$$



A	B	Diff	Bo
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Full Subtractor

Full Subtractor

$$\text{Bout} = \sum m(1,2,3,7)$$

AB/ Bin	00	01	11	10
0	0	1	0	0
1	1	1	1	0

$$\text{Bout} = A'B + A'Bin + B Bin$$

$$\text{Sum} = \sum m(1,2,4,7)$$

$$= A \text{ xor } B \text{ xor } Bin$$

Full Subtractor Truth Table

A	B	Bin	Diff	Bo
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Full Subtractor

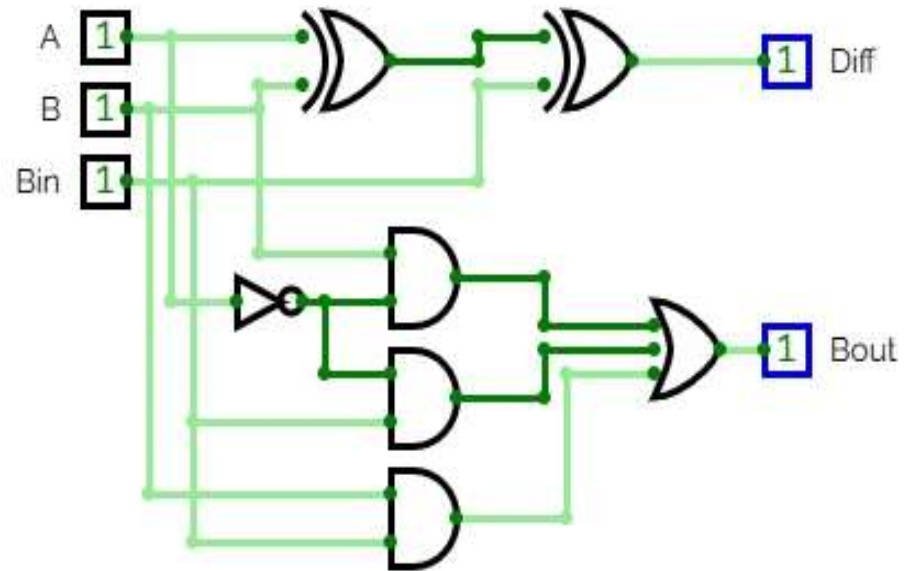
Full Subtractor

$$\text{Bout} = \sum m(1,2,3,7)$$

$$\text{Bout} = A'B + A'Bin + B Bin$$

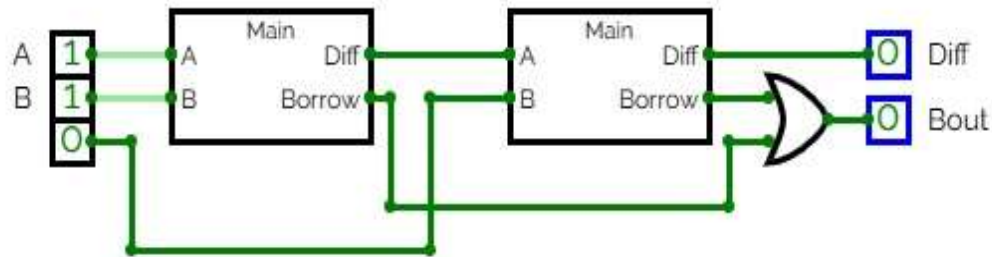
$$\text{Sum} = \sum m(1,2,4,7)$$

$$= A \text{ xor } B \text{ xor } Bin$$



Full Subtractor using Half Subtractors

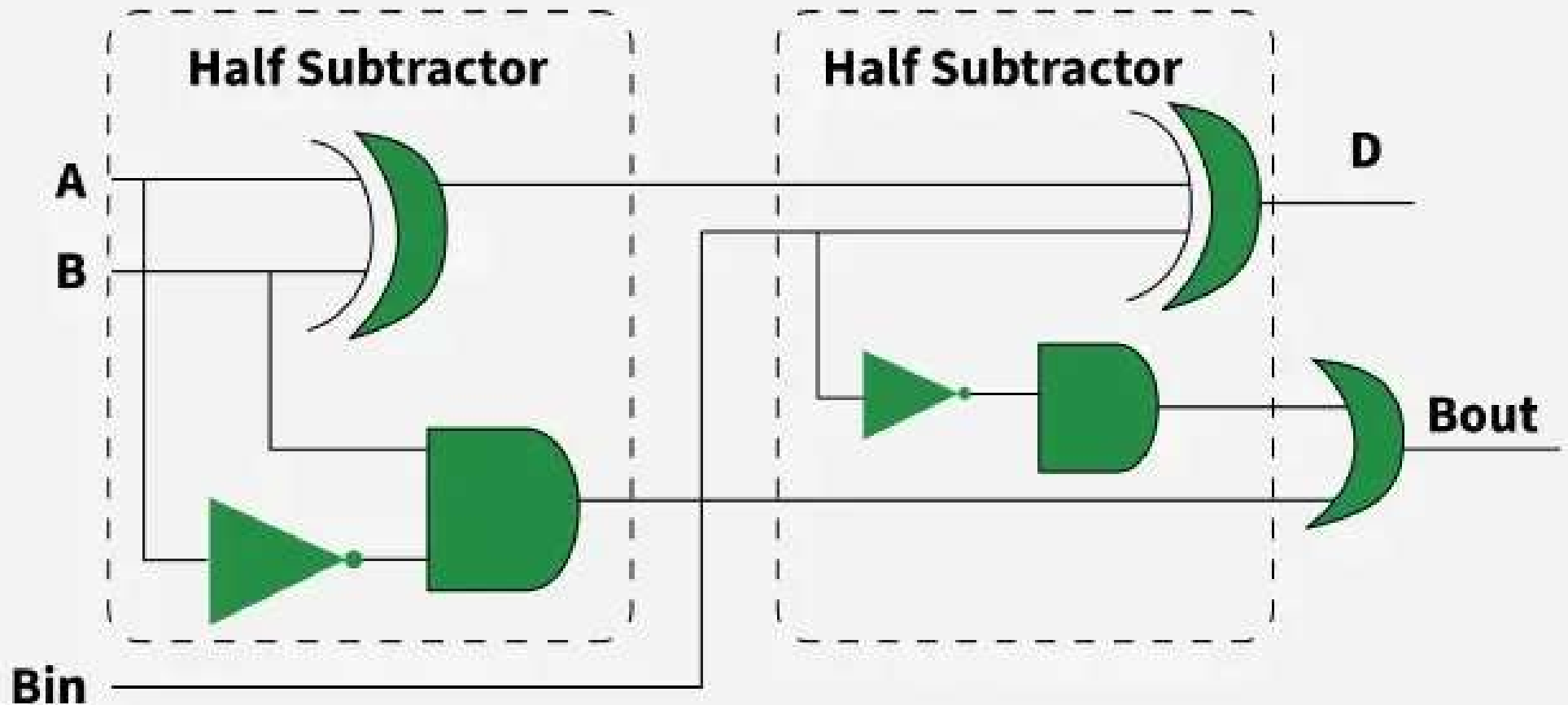
$$\text{Diff} = A \oplus B \oplus \text{Bin}$$
$$\text{Bout} = A'B + A'\text{Bin} + B\text{Bin}$$



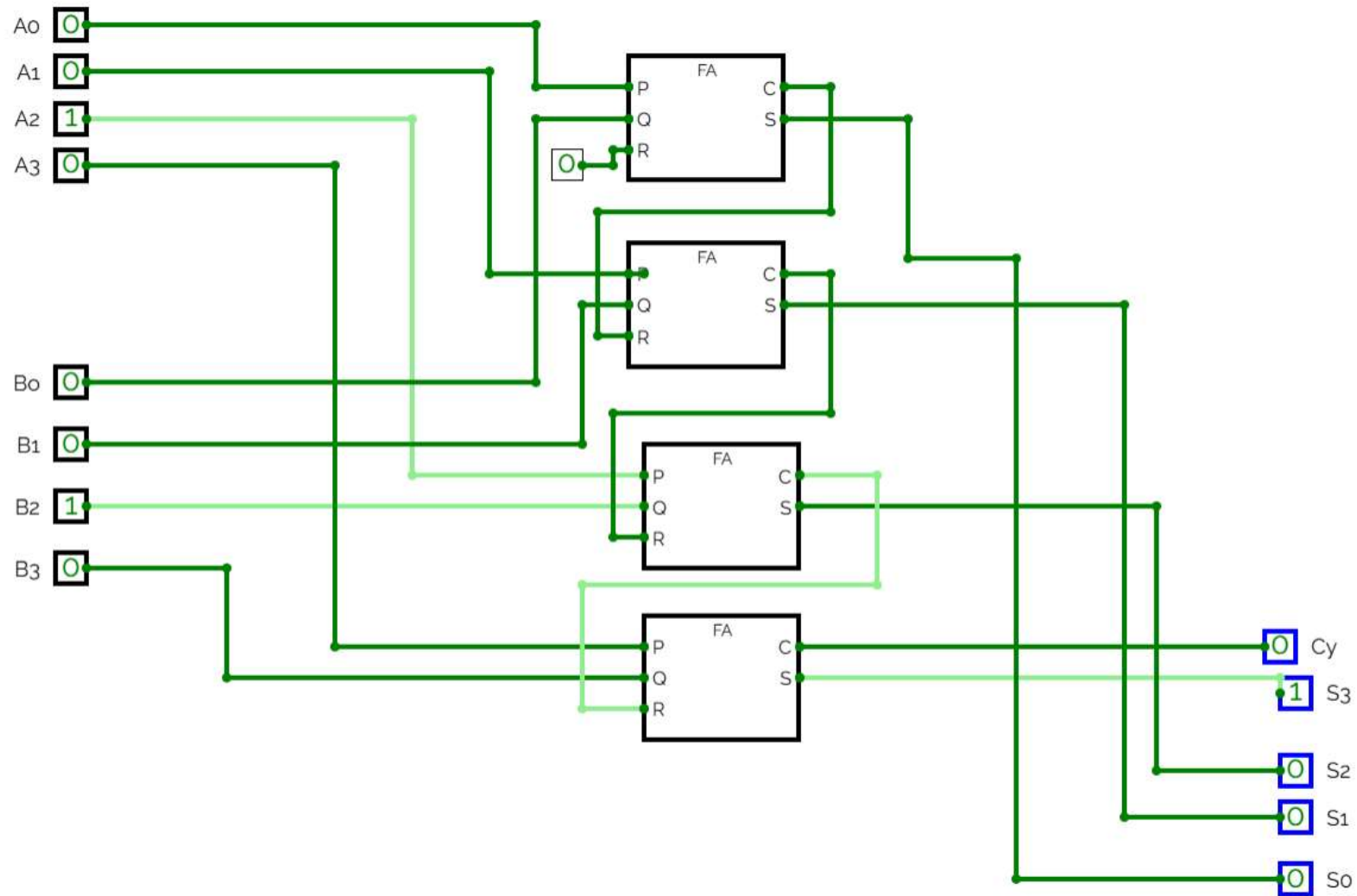
Full Subtractor Truth Table

A	B	Bin	Diff	Bo
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

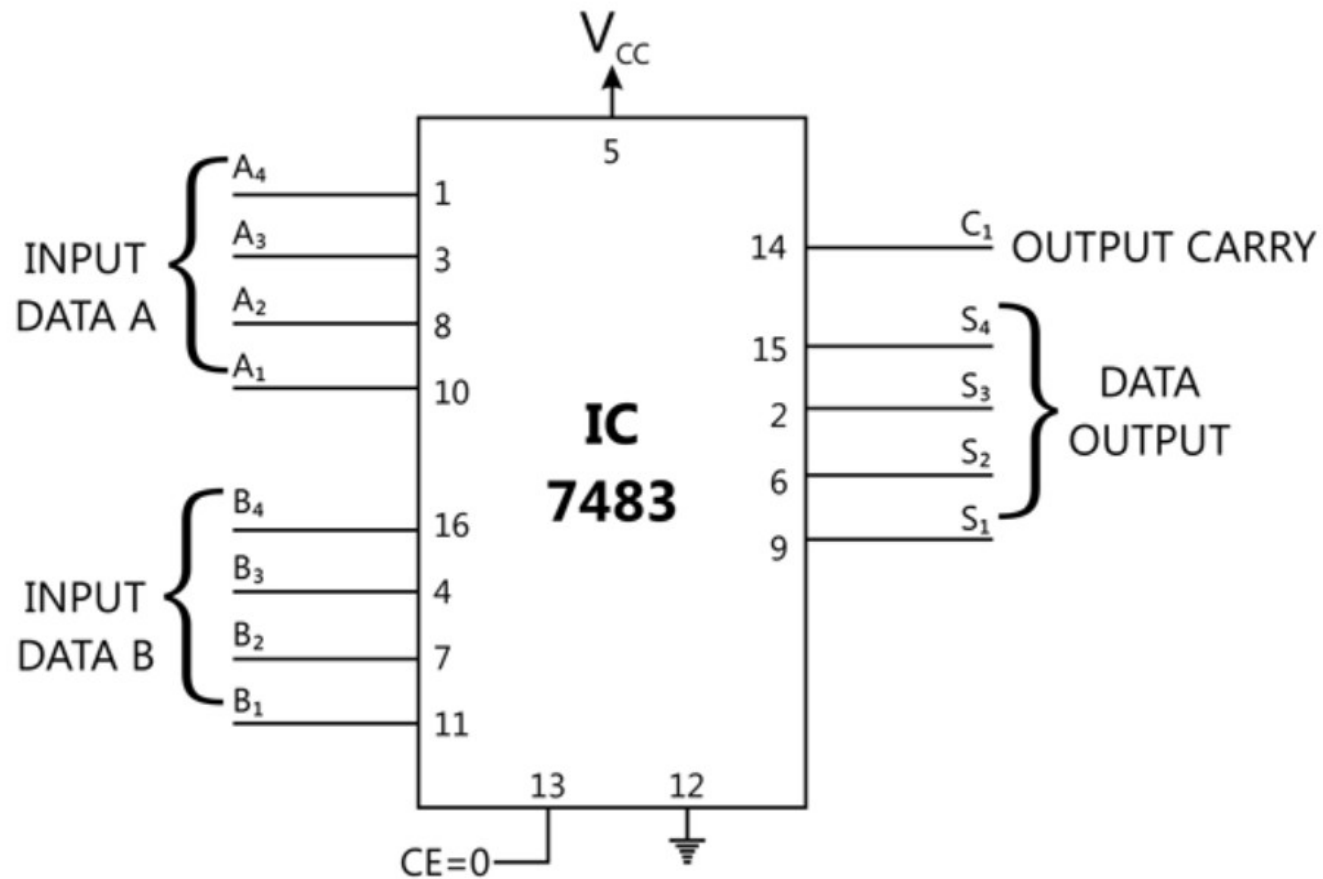
Full Subtractor using Half Subtractors



4 bit adder using full Adder



4 bit adder IC 7483



Controlled Adder Subtractor

Circuit which can be Controlled as adder as well as subtractor

If control input $C = 0$...Adder

If Control input $C = 1$...Subtractor

Addition

	A3	A2	A1	A0
+	B3	B2	B1	B0
C3	C2	C1	C0	
Cout	S3	S2	S1	S0

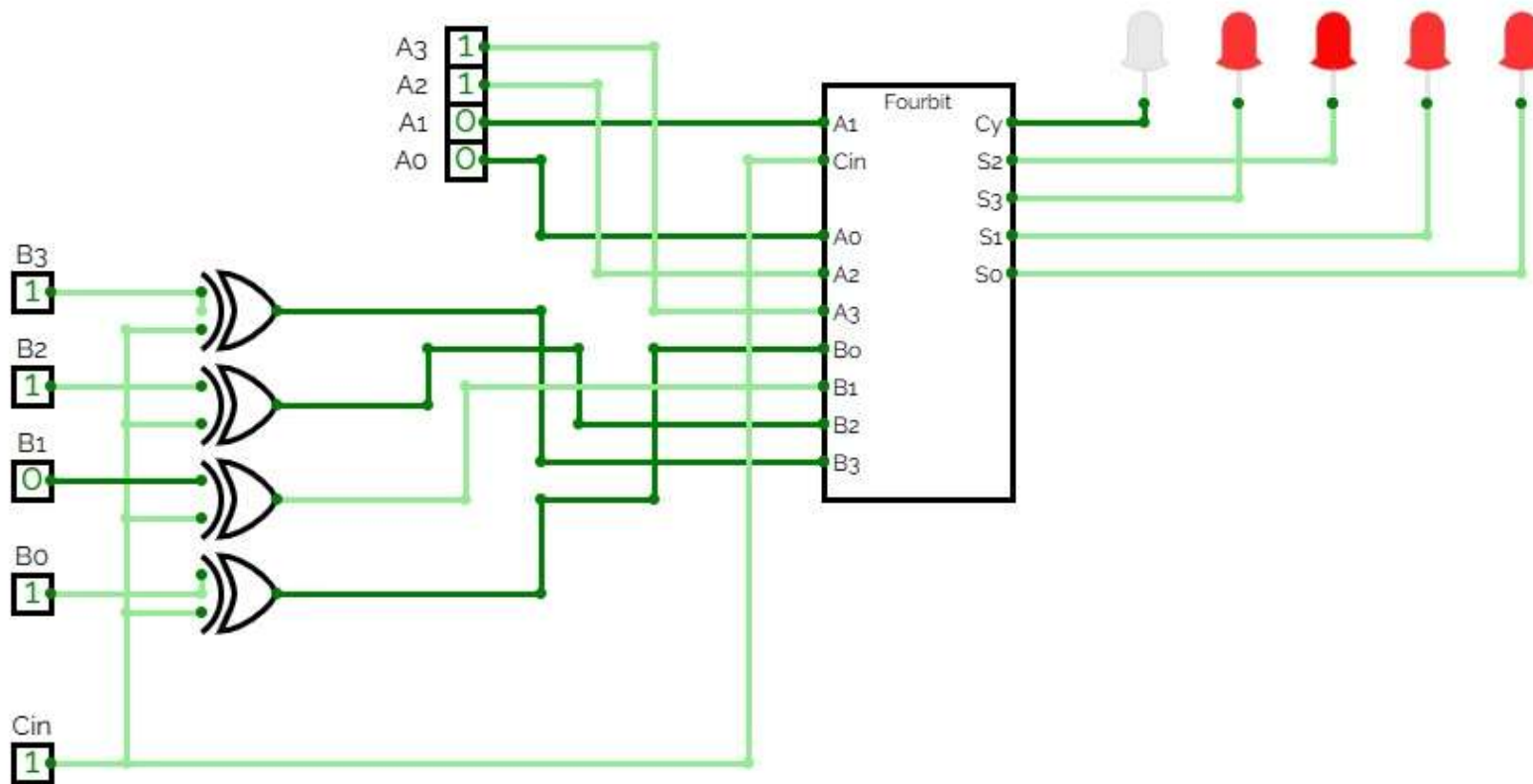
Subtraction: by 2's complement

	A3	A2	A1	A0
+	$B3'$	$B2'$	$B1'$	$B0'$
C3	C2	C1	C0	1
Cout	S3	S2	S1	S0

1's complement of B can be obtained by 4 XOR gates as

$1 \text{ xor } A = A'$, $0 \text{ xor } A = A$

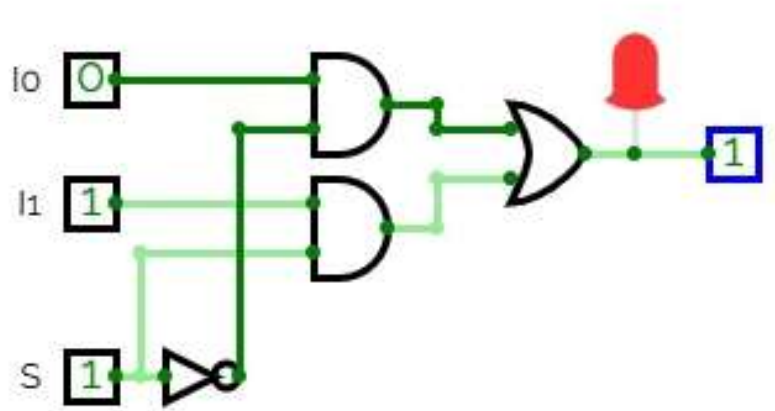
Controlled Adder Subtractor



2:1 Multiplexers

$$Y = S0'.I0 + S0. I1$$

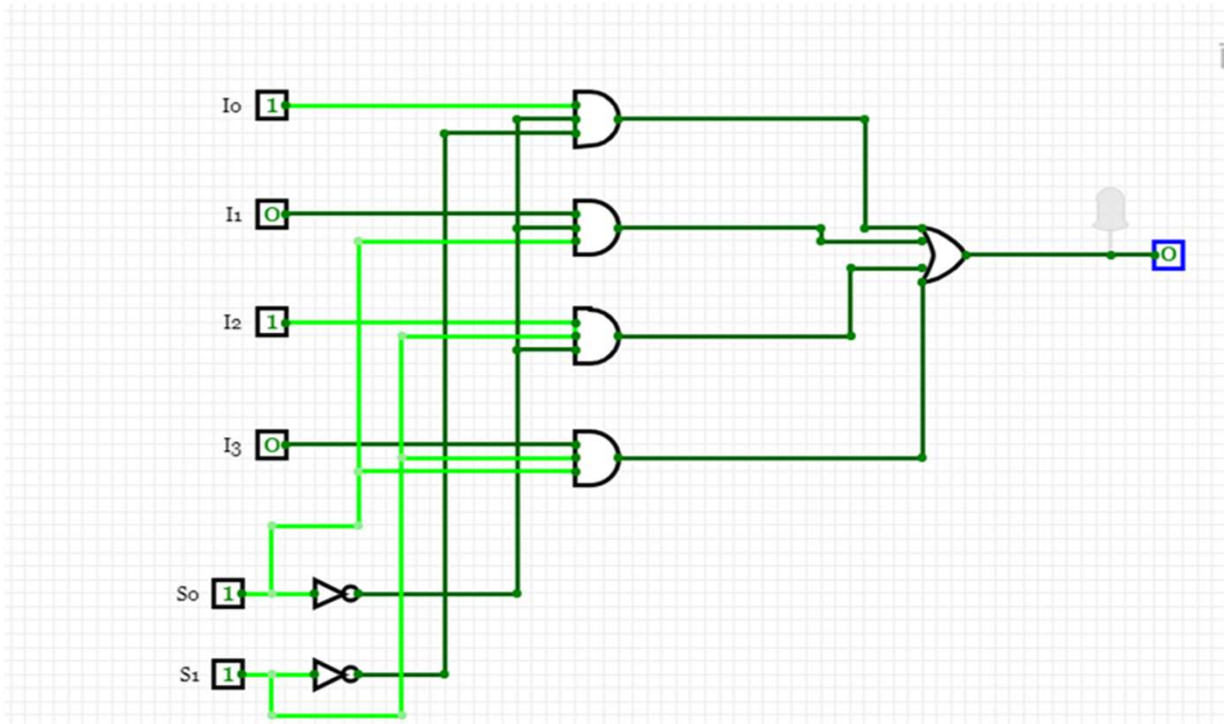
S0	Y
0	I0
1	I1



4:1 Multiplexers

4:1 Multiplexer

$$Y = S1'.S0'.I0 + S1'.S0.I1 + S1.S0'.I2 + S1.S0.I3$$

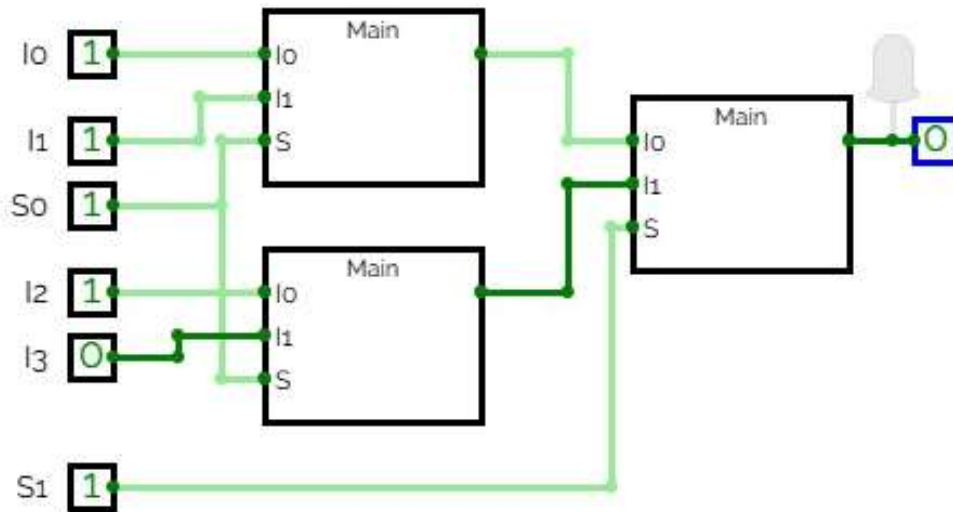


S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

4:1 Multiplexers using 2:1 Mux

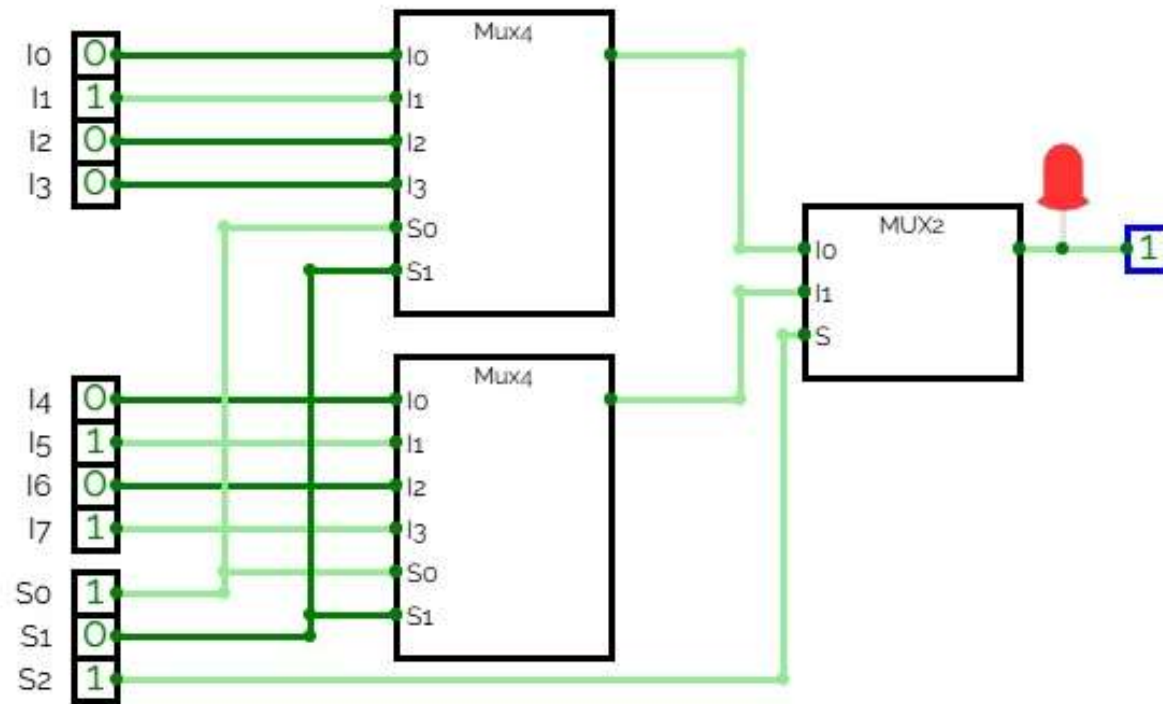
4:1 Multiplexer

$$Y = S1'.S0'.I0 + S1'.S0.I1 + S1.S0'.I2 + S1.S0.I3$$

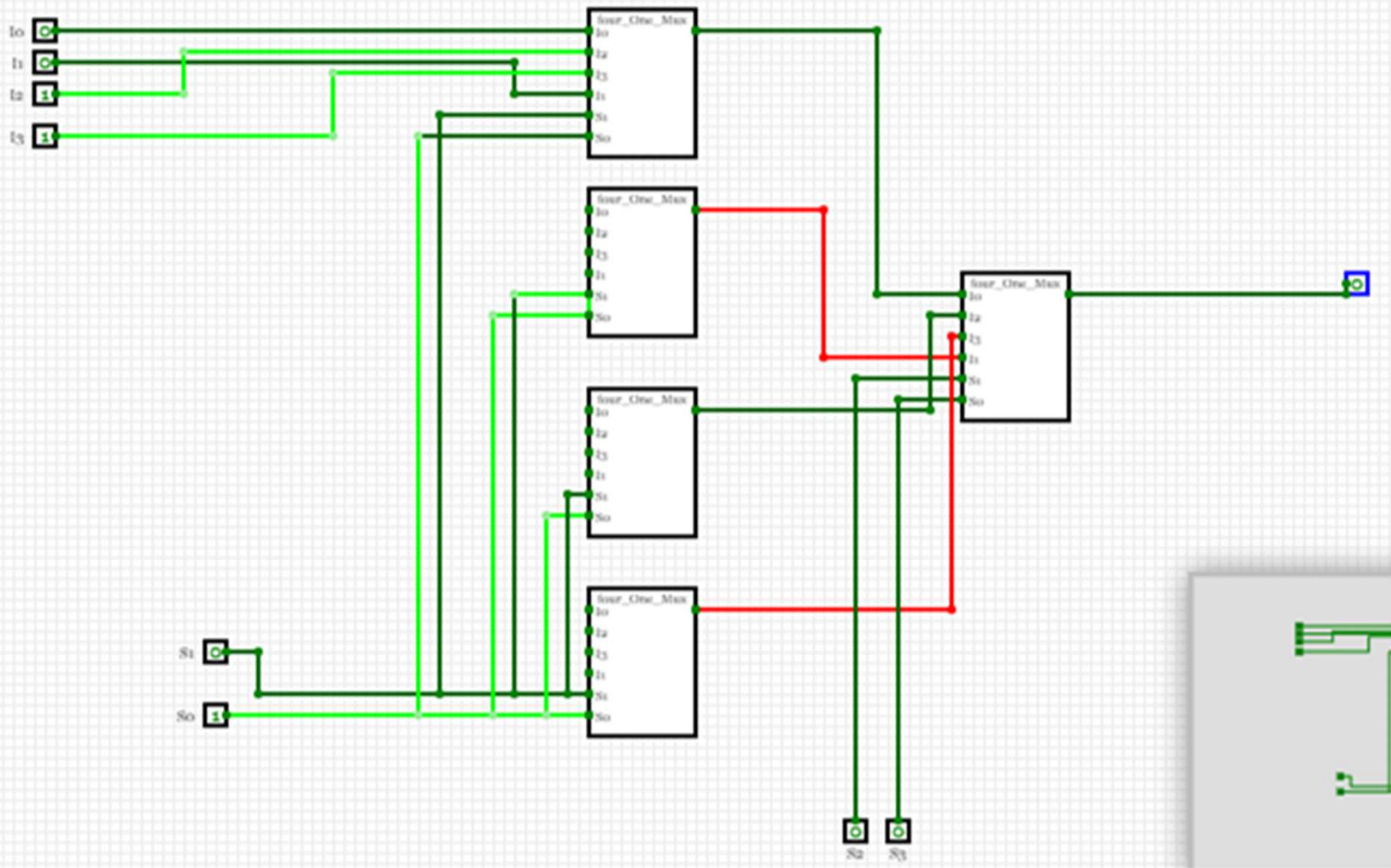


S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

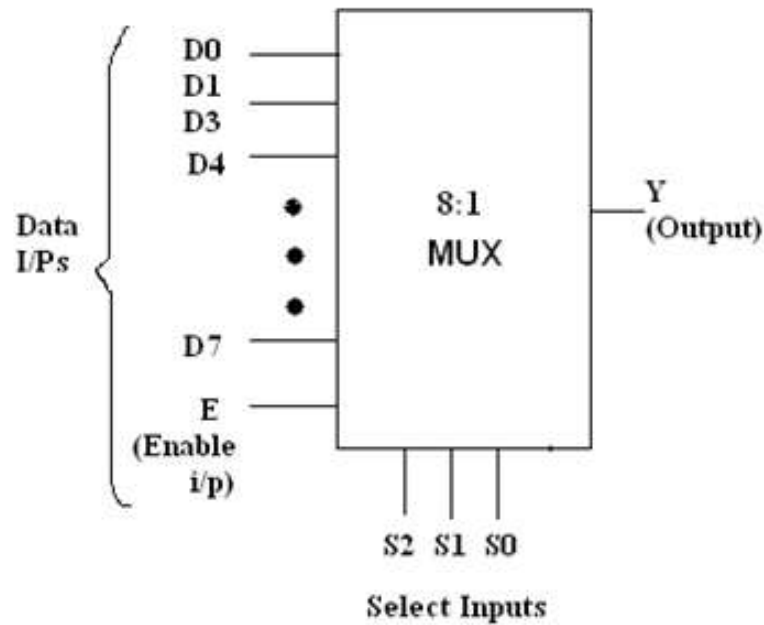
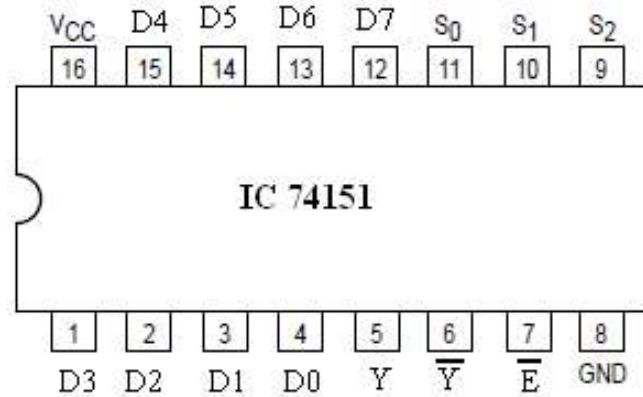
Multiplexer Tree : 8:1 using 4:1 Mux



Multiplexer Tree ...16:1 using 4:1 Mux

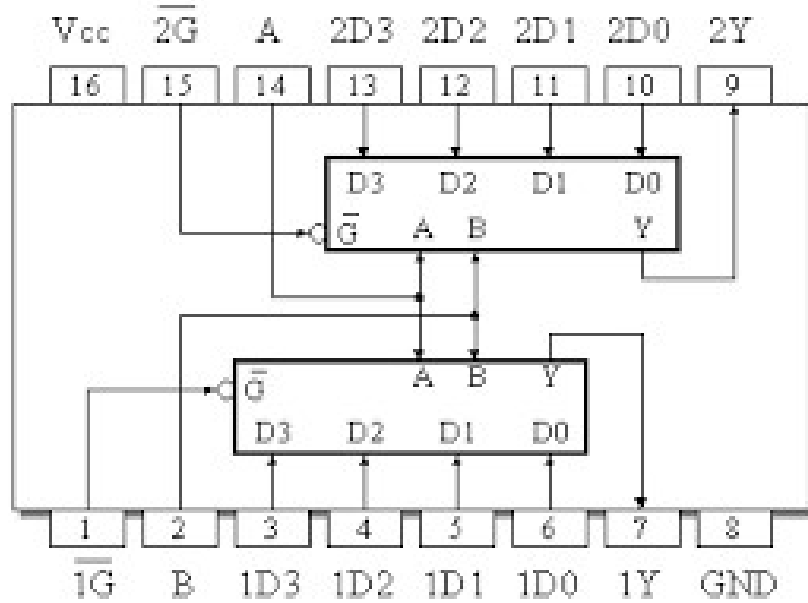


Multiplexers 8:1 : IC 74151



Enable	Select Inputs			Output
E	S ₂	S ₁	S ₀	Y
0	X	X	X	0
1	0	0	0	D ₀
1	0	0	1	D ₁
1	0	1	0	D ₂
1	0	1	1	D ₃
1	0	0	0	D ₄
1	0	0	1	D ₅
1	0	1	0	D ₆
1	0	1	1	D ₇

Dual 4:1 Mux IC 74153



Truth Table

Inputs			Output
B	A	G'	Y
X	X	1	0
0	0	0	D0
0	1	0	D1
1	0	0	D2
1	1	0	D3

Decoder

Truth table

D1	D0	O3	O2	O1	O0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Minterm Generator

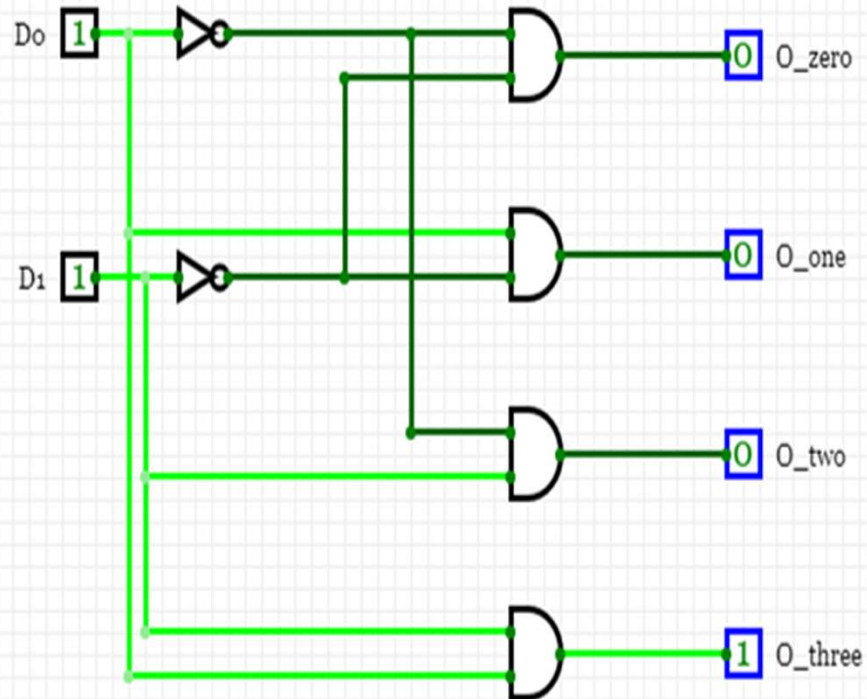
$$O0 = D1'D0'$$

$$O1 = D1'D0$$

$$O2 = D1D0'$$

$$O3 = D1D0$$

2:4 Decoder



Decoder with active low output

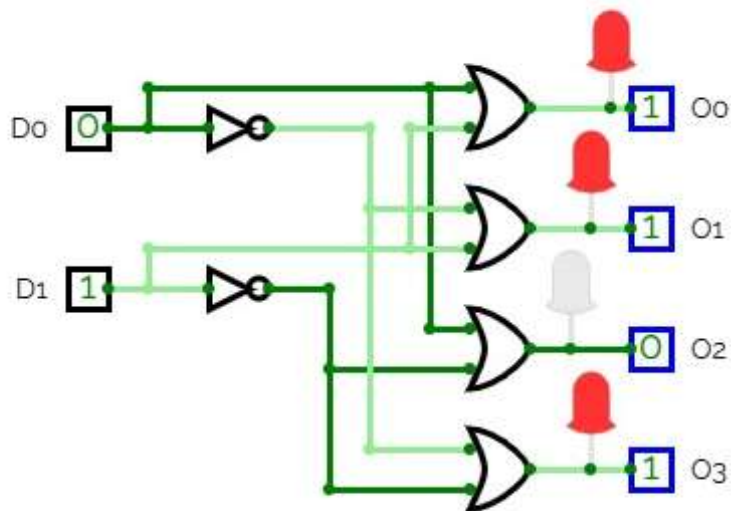
Maxterm Generator

$$O0 = (D1 + D0)$$

$$O1 = (D1 + D0')$$

$$O2 = (D1' + D0)$$

$$O3 = (D1' + D0')$$



D1	D0	O3	O2	O1	O0
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

Decoder with enable input

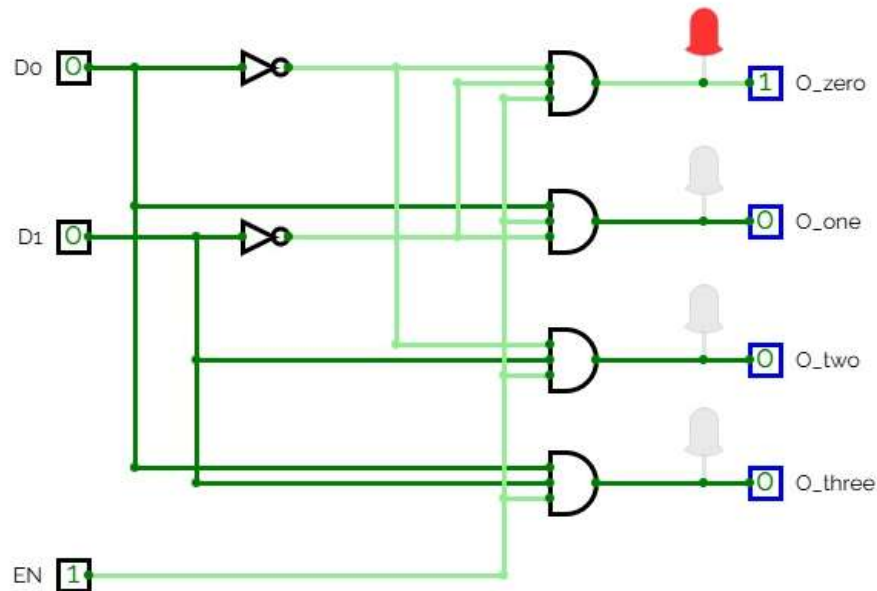
Minterm Generator

$$O0 = En \cdot D1' \cdot D0'$$

$$O1 = En \cdot D1' \cdot D0$$

$$O2 = En \cdot D1 \cdot D0'$$

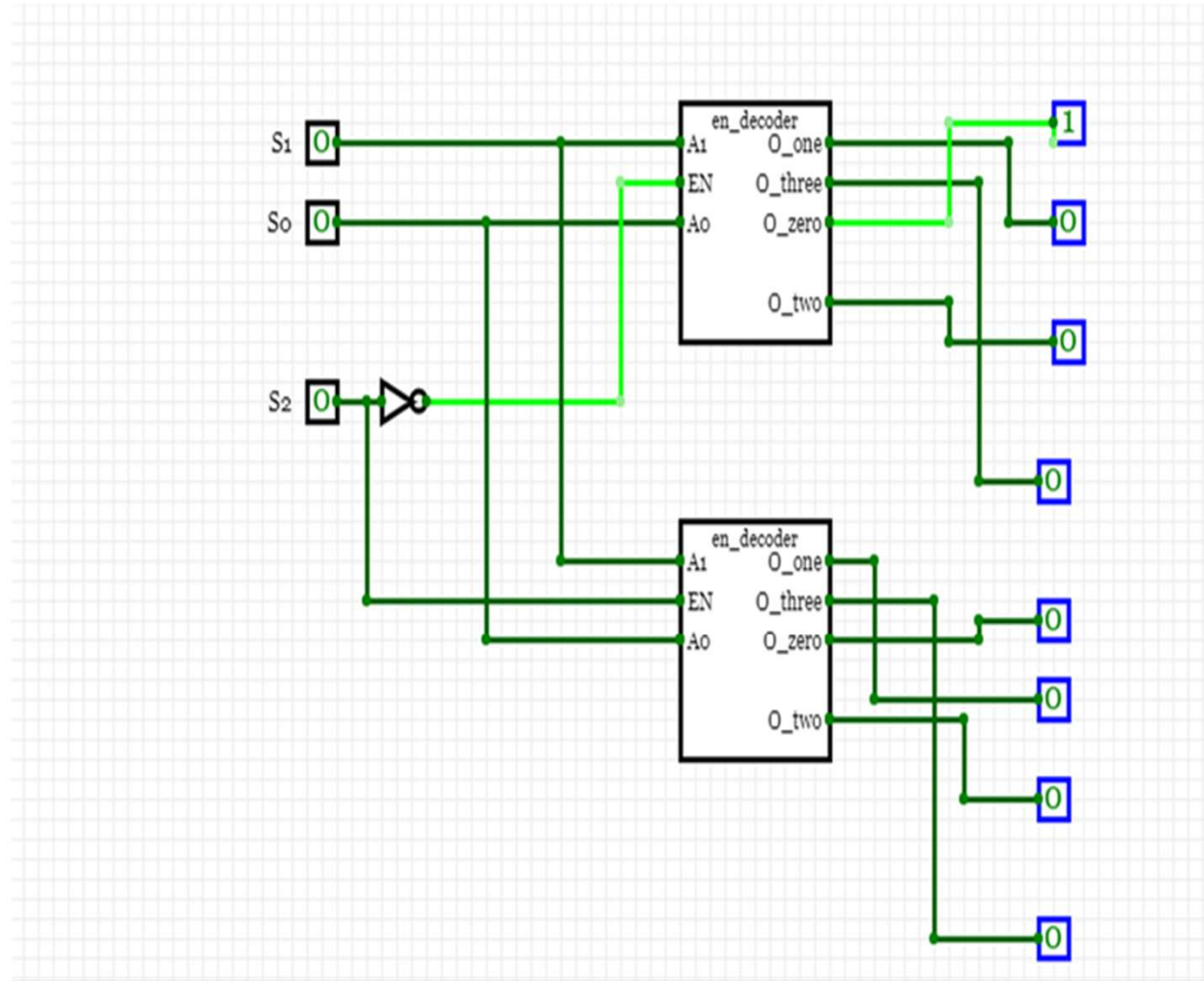
$$O3 = En \cdot D1 \cdot D0$$



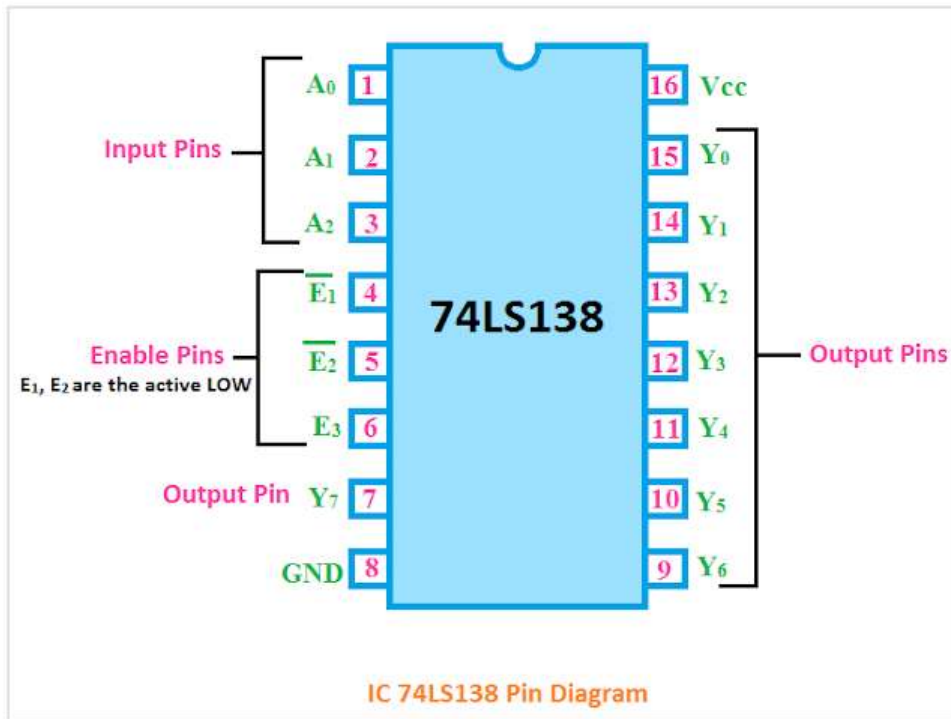
En	D1	D0	O3	O2	O1	O0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Decoder Tree

3:8 decoder using 2:4



IC 74138: 3:8 Decoder



INPUTS						OUTPUTS							
$\overline{E_1}$	$\overline{E_2}$	E ₃	A ₀	A ₁	A ₂	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

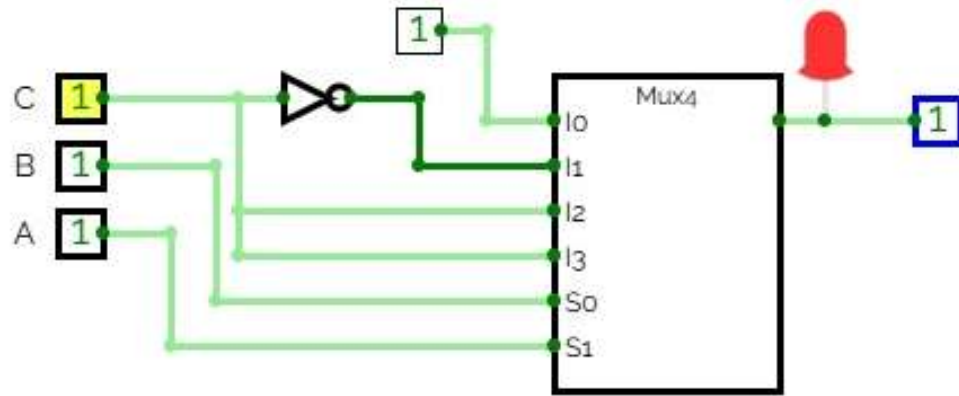
H - High, L - Low, X - Don't Care

IC 74138 Truth Table

Function Implementation using Multiplexer

$$F = \sum m(0,1,2,5,7)$$

A	B	C	Y	
0	0	0	1	1
0	0	1	1	
0	1	0	1	C'
0	1	1	0	
1	0	0	0	C
1	0	1	1	
1	1	0	0	C
1	1	1	1	



Function Implementation using Multiplexer

$F = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$

Implement using

8:1 mux

Implement using

4:1 mux

Using 8:1 Mux

ABC are select Lines

A	B	C	D	F		A	B	C	D	F	
0	0	0	0	1	D'	1	0	0	0	1	D'
0	0	0	1	0		1	0	0	1	0	
0	0	1	0	1	D'	1	0	1	0	1	D'
0	0	1	1	0		1	0	1	1	0	
0	1	0	0	0	D	1	1	0	0	0	D
0	1	0	1	1		1	1	0	1	1	
0	1	1	0	0	D	1	1	1	0	0	D
0	1	1	1	1		1	1	1	1	1	

Function Implementation using Multiplexer

$$F = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$$

Implement using 4:1
mux

Use select Lines as AB

AB / CD	00	01	10	11
00	1		1	
01		1		1
10	1		1	
11		1		1
	D'	D	D'	D

Function Implementation using Multiplexer

F =
 $\Sigma m(0,1,4,6,8,10,12,15)$
)

**Implement using 8:
 1 mux**

**Implement using 4:1
 mux**

Using 8:1 Mux
 ABC are select Lines

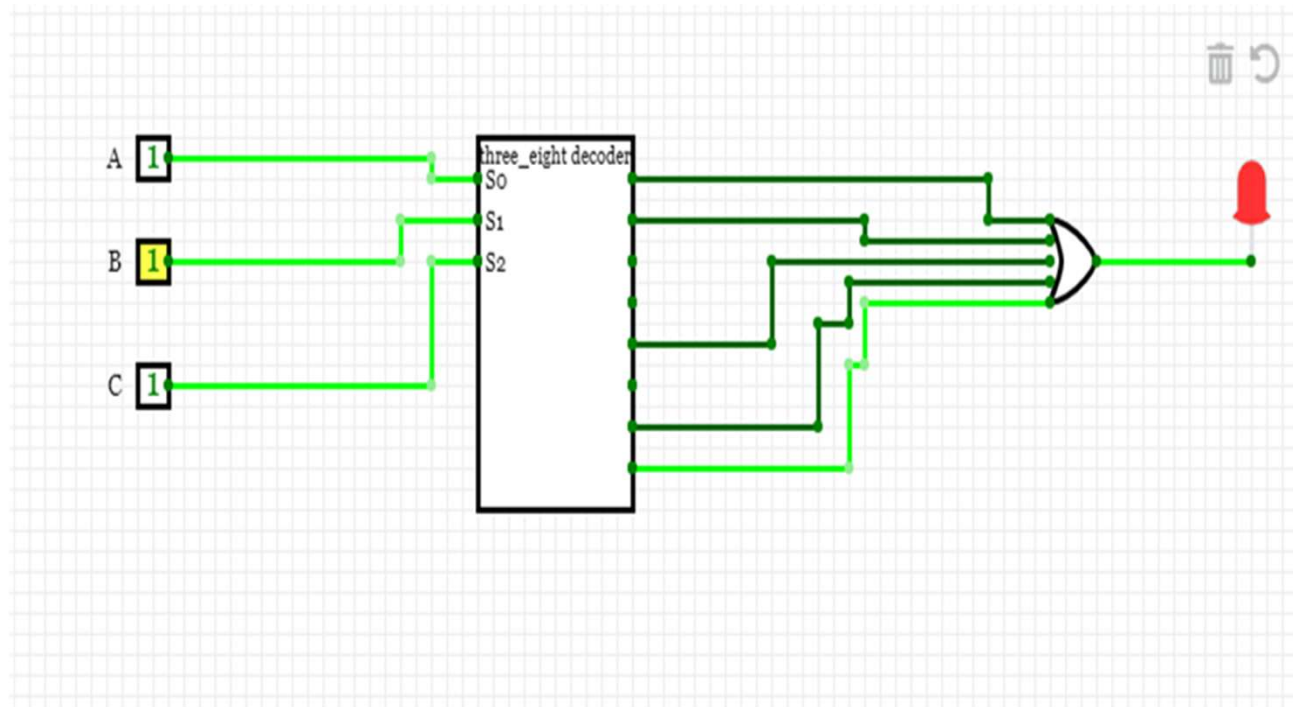
A	B	C	D	F	
0	0	0	0		
0	0	0	1		
0	0	1	0		
0	0	1	1		
0	1	0	0		
0	1	0	1		
0	1	1	0		
0	1	1	1		

A	B	C	D	F	
1	0	0	0		
1	0	0	1		
1	0	1	0		
1	0	1	1		
1	1	0	0		
1	1	0	1		
1	1	1	0		
1	1	1	1		

Function Implementation using Decoder

$$F = \sum m(0, 1, 4, 6, 7)$$

Implement using 3: 8 decoder(Active High Output)



Function Implementation using Decoder

$F = \sum m(0,1,4,6,8,10,12,15)$

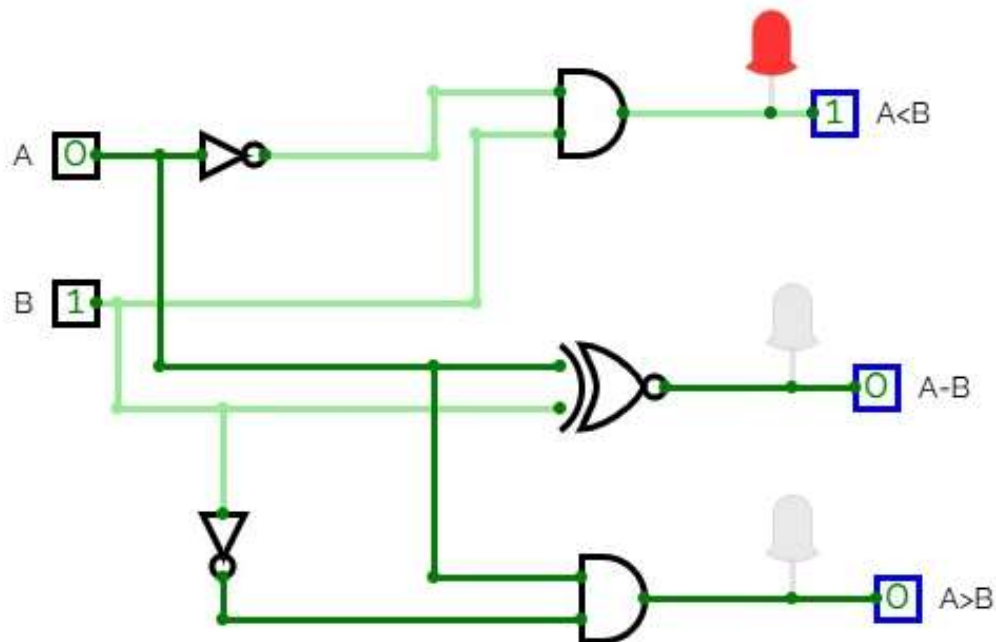
Implement 4: 16 decoder

1 bit Comparator

$$(A < B) = A'B$$

$$(A = B) = A \text{ XNOR } B$$

$$(A > B) = AB'$$



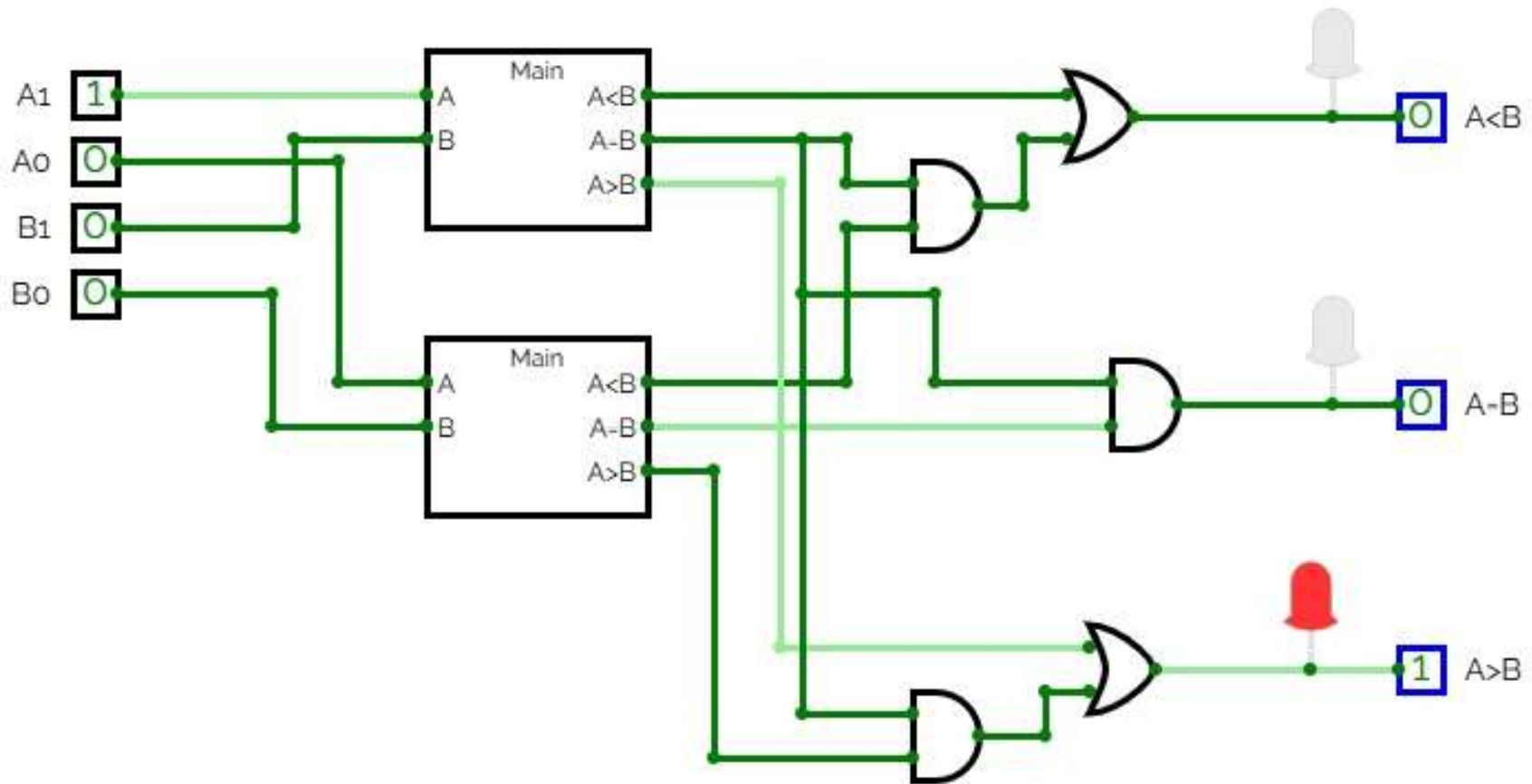
A	B	A < B	A = B	A > B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

2 bit Comparator

A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0

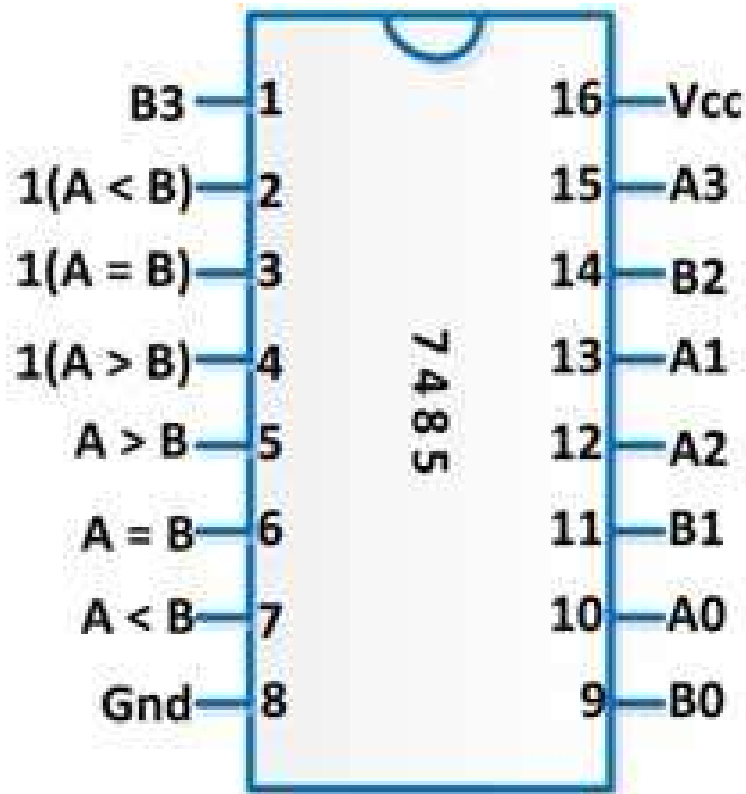
A1	A0	B1	B0	A<B	A=B	A>B
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

2 bit Comparator



4 bit Comparator

IC 7485 comparator



Pin Number	Pin Name	Pin Function
1	B3	MSB of Binary Number B
2	A < Bin	Input for A < B on cascading
3	A = Bin	Input for A = B on cascading
4	A > Bin	Input for A > B on cascading
5	A > Bout	Comparator Output for A > B
6	A = Bout	Comparator Output for A = B
7	A < Bout	Comparator Output for A < B
8	GND	Ground
9	B0	LSB of Binary Number B
10	A0	LSB of Binary Number A
11	B1	Third MSB of Binary Number B
12	A1	Third MSB of Binary Number A
13	A2	Second MSB of Binary Number A
14	B2	Second MSB of Binary Number B
15	A3	MSB of Binary Number A
16	VCC	Supply Voltage

4 bit Comparator

Comparing Inputs				Cascading Inputs			Outputs		
A ₃ , B ₃	A ₂ , B ₂	A ₁ , B ₁	A ₀ , B ₀	I (A > B)	I (A < B)	I (A = B)	Y (A > B)	Y (A < B)	Y (A = B)
A ₃ > B ₃	X	X		X	X	X	H	L	L
A ₃ < B ₃	X	X	X	X	X	X	L	H	L
A ₃ = B ₃	A ₂ > B ₂	X	X	X	X	X	H	L	L
A ₃ = B ₃	A ₂ < B ₂	X	X	X	X	X	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ > B ₁	X	X	X	X	H	L	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ < B ₁	X	X	X	X	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ > B ₀	X	X	X	H	L	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ < B ₀	X	X	X	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	H	L	L	H	L	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	L	H	L	L	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	L	L	L	H	H	L
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	X	X	H	L	L	H
A ₃ = B ₃	A ₂ = B ₂	A ₁ = B ₁	A ₀ = B ₀	H	H	L	L	L	L

Comparing Inputs	Cascading Inputs			outputs		
	I (A<B)	I (A=B)	I (A>B)	Y (A<B)	Y (A=B)	Y (A>B)
A < B	X	X	X	1	0	0
A = B	0	0	0	1	0	1
	0	0	1	0	0	1
	1	0	0	1	0	0
	1	0	1	0	0	0
	X	1	X	0	1	0
A > B	X	X	X	0	0	1

Encoders



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



Definition

An **encoder** is a combinational circuit that converts **2^n input lines into n output lines**. Only **one input should be HIGH (1)** at a time.

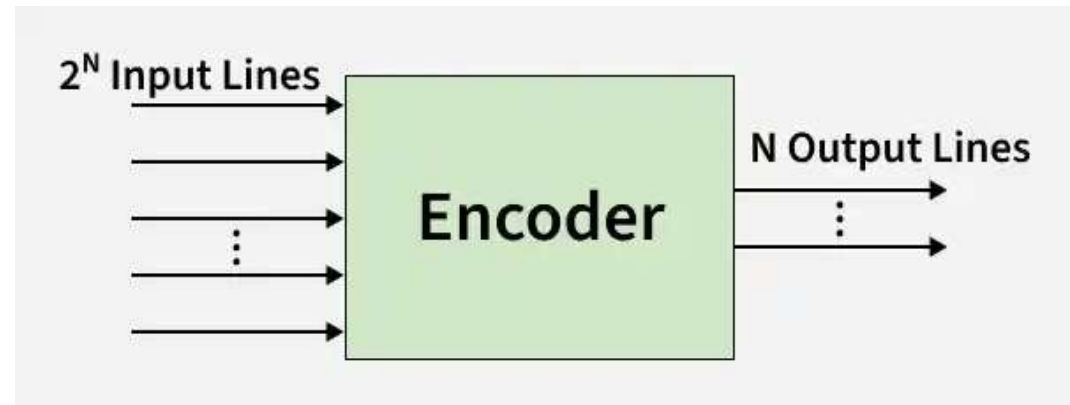
Example:

- 4-to-2 encoder $\rightarrow$ 4 inputs, 2 outputs
- 8-to-3 encoder $\rightarrow$ 8 inputs, 3 outputs

Example: 4-to-2 Encoder

Inputs: Y0, Y1, Y2, Y3

Outputs: A1, A0



Active Input	A1	A0
Y0 = 1	0	0
Y1 = 1	0	1
Y2 = 1	1	0
Y3 = 1	1	1

- **Boolean Expressions:**

$$A1 = Y2 + Y3$$

$$A0 = Y1 + Y2$$

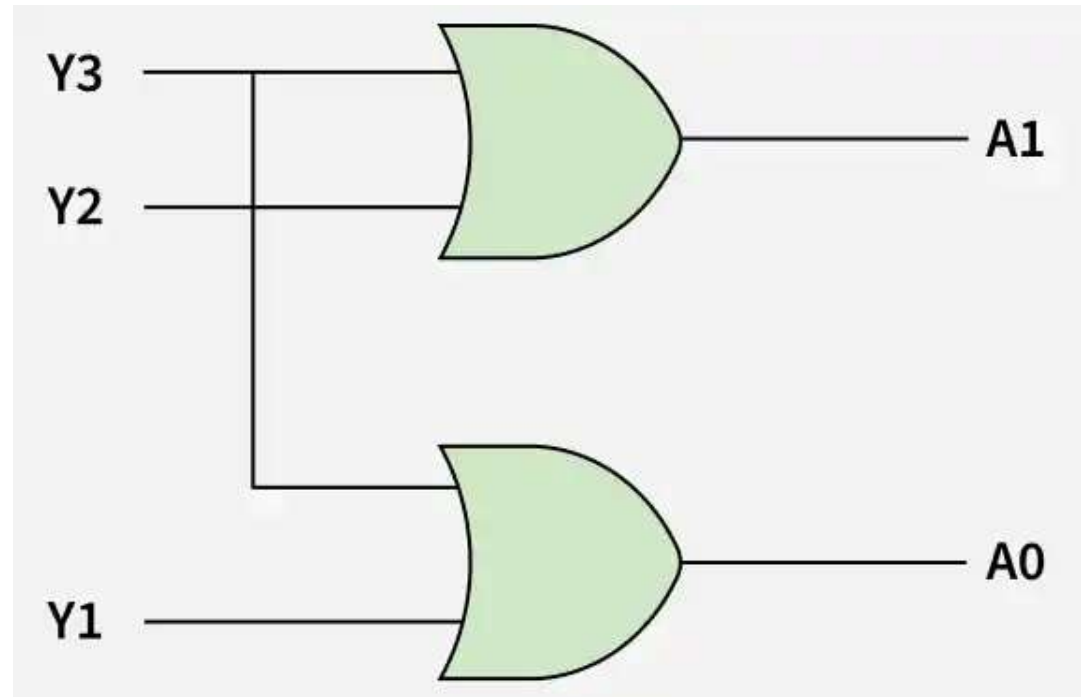
- **Applications:**

Keyboard encoding

Interrupt handling

Data compression

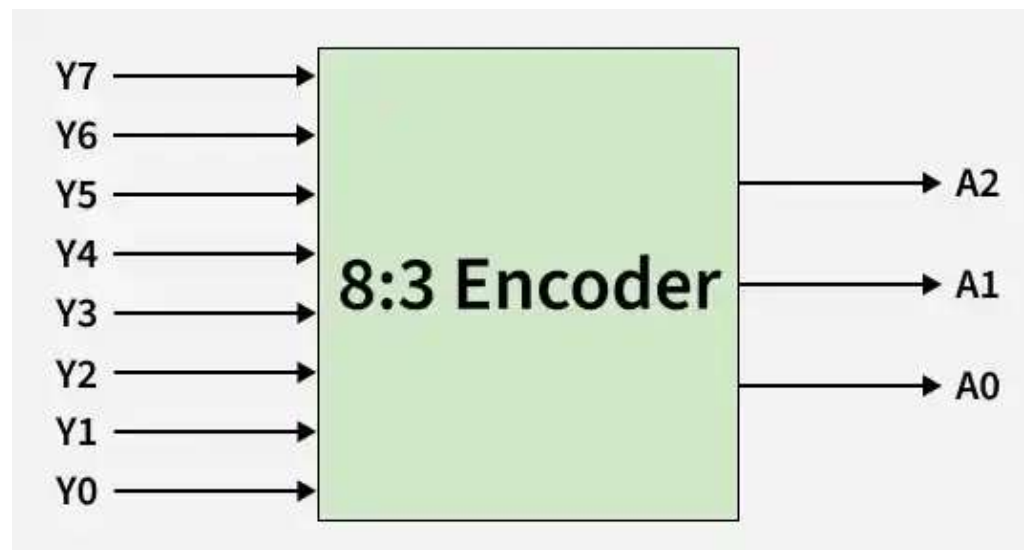
Microprocessor input systems



Octal to Binary Encoder (8 to 3 Encoder)

The 8 to 3 Encoder or octal to Binary encoder consists of 8 inputs: Y7 to Y0 and 3 outputs: A2, A1 & A0. Each input line corresponds to each octal digit value and three outputs generate corresponding binary code.

The figure below shows the logic symbol of octal to the binary encoder.



Truth Table

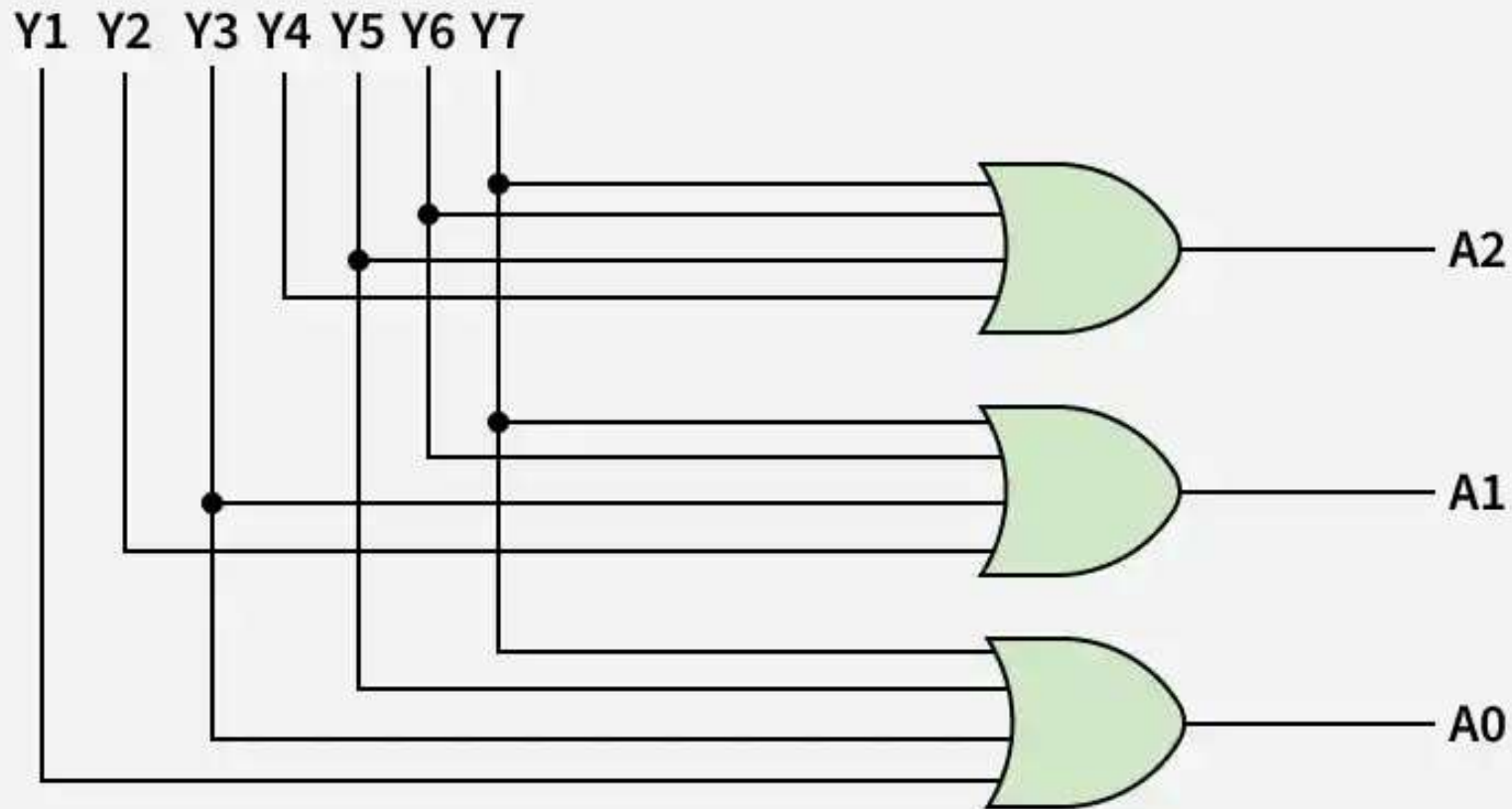
INPUTS($Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$)	OUTPUTS($A_2A_1A_0$)
00000001	000
00000010	001
00000100	010
00001000	011
00010000	100
00100000	101
01000000	110
10000000	111

Logical expression for A_2 , A_1 , and A_0 :

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

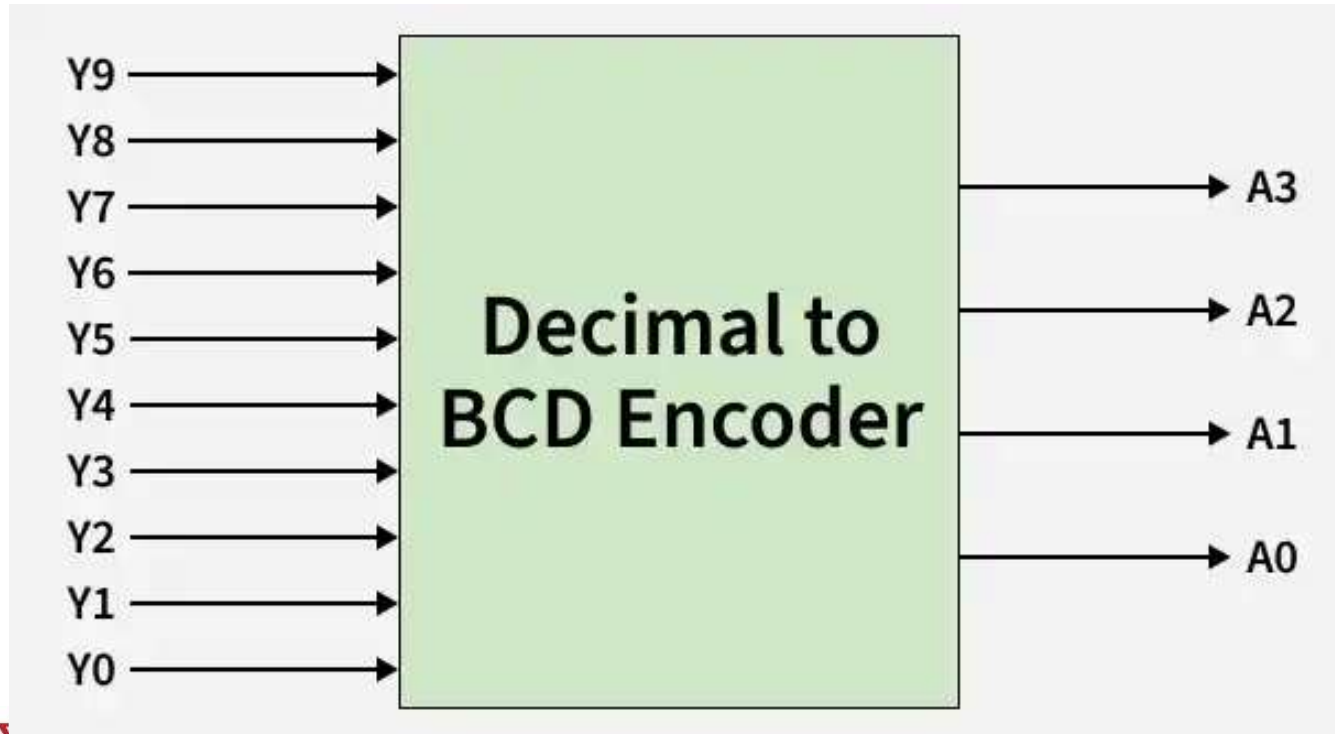
$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$



Decimal to BCD Encoder (16:4)

The decimal-to-binary encoder usually consists of 16 input lines and 4 output lines. Each input line corresponds to each decimal digit and 4 outputs correspond to the BCD code. This encoder accepts the decoded decimal data as an input and encodes it to the BCD output which is available on the output lines. The figure below shows the logic symbol of the decimal to BCD encoder.



Truth Table

INPUTS($Y_9Y_8Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$)	OUTPUTS($A_3A_2A_1A_0$)
0000000001	0000
0000000010	0001
0000000100	0010
0000001000	0011
0000010000	0100
0000100000	0101
0001000000	0110
0010000000	0111
0100000000	1000
1000000000	1001

Logical expression for A3, A2, A1, and A0:

$$A3 = Y15 + Y14 + Y13 + Y12 + Y11 + Y10 + Y9 + Y8$$

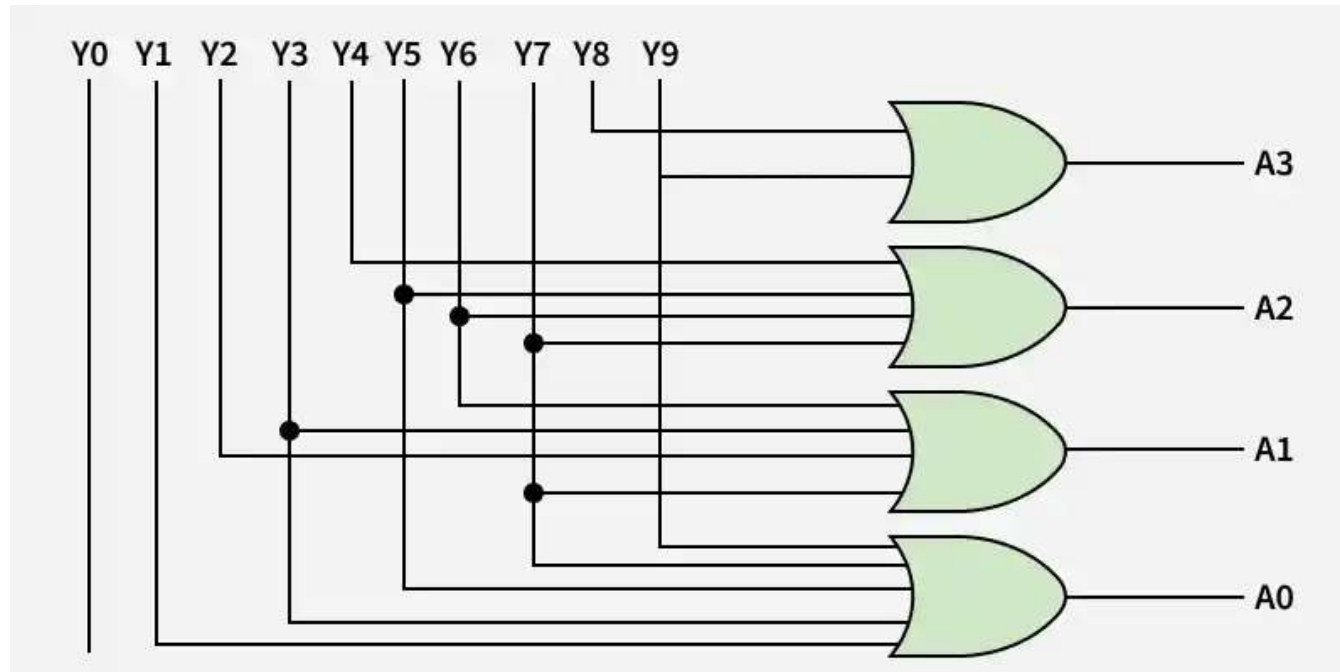
$$A2 = Y15 + Y14 + Y13 + Y12 + Y7 + Y6 + Y5 + Y4$$

$$A1 = Y15 + Y14 + Y11 + Y10 + Y7 + Y6 + Y3 + Y2$$

$$A0 = Y15 + Y13 + Y11 + Y9 + Y7 + Y5 + Y3 + Y1$$

Circuit Diagram

The above two Boolean functions can be implemented using OR gates:



Priority Encoders



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



Definition

A **priority encoder** is an encoder that assigns **priority to inputs**.

If **multiple inputs are HIGH at the same time**, the encoder outputs the code of the **highest priority input**.

Usually, the highest-numbered input has the highest priority. A 4 to 2 priority encoder has 4 inputs: Y_3 , Y_2 , Y_1 & Y_0 , and 2 outputs: A_1 & A_0 . Here, the input, Y_3 has the highest priority, whereas the input, Y_0 has the lowest priority. In this case, even if more than one input is '1' at the same time, the output will be the (binary) code corresponding to the input, which is having higher priority.

Example: 4-to-2 Priority Encoder

Priority Order:

$Y_3 > Y_2 > Y_1 > Y_0$

(x = don't care)

Inputs	Output ($A_1 A_0$)
1xxx	11
01xx	10
001x	01
0001	00

Logical Expressions using K-Map

The logical expression for A_1 is shown below:

Y3 Y2 \ Y1 Y0		00	01	11	10
		00	01	11	10
00	x	0	0	0	0
01	1	1	1	1	1
11	1	1	1	1	1
10	1	1	1	1	1

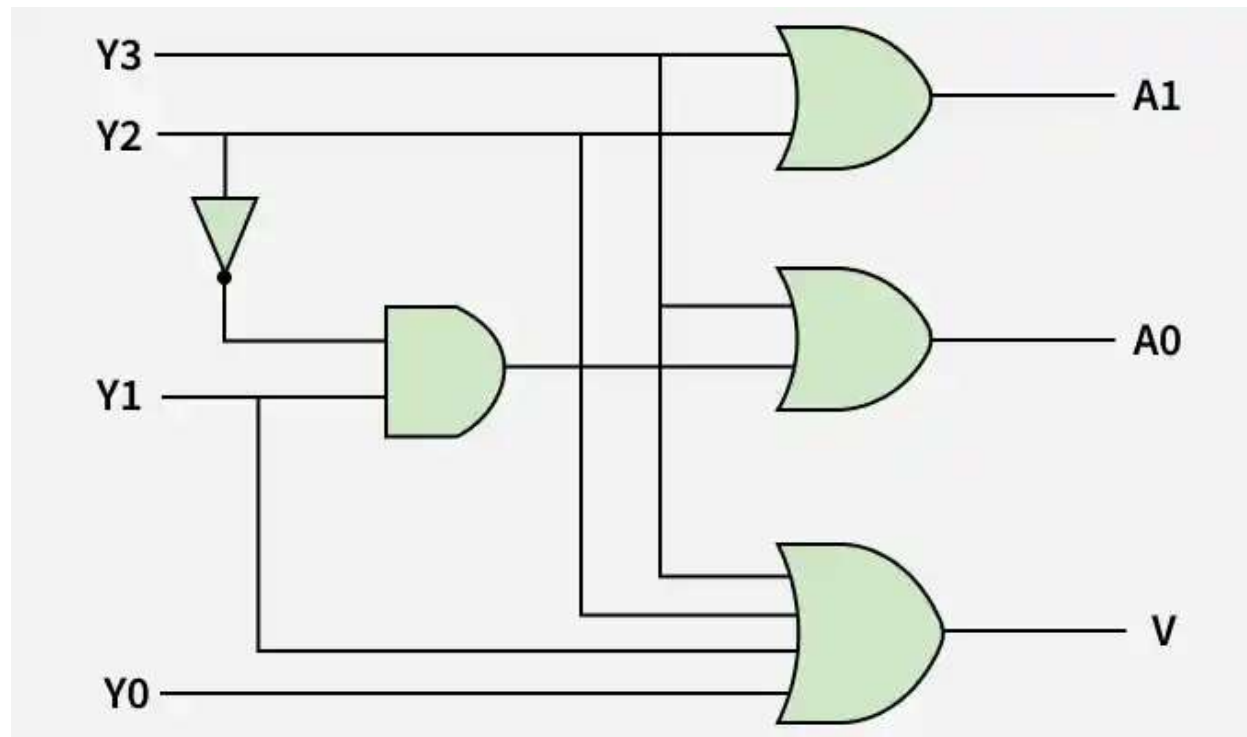
$A_1 = Y_3 + Y_2$

The Logical Expression for A_0 is shown below:

Y3 Y2 \ Y1 Y0		00	01	11	10
		00	01	11	10
00	x	0	1	1	
01	0	0	0	0	
11	x	x	x	x	
10	1	1	1	1	

$A_0 = Y_3 + Y_3'Y_1$

Circuit Diagram: The above two Boolean functions can be implemented as:



Applications of Priority Encoder:

Interrupt controllers

CPU scheduling

Bus arbitration

Resource allocation systems

De- multiplexer (DEMUX)



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



De-multiplexer (DEMUX)

- The DEMUX is a digital information processor. It takes input from one source and also converts the data to transmit towards various sources. The demultiplexer has one data input line.
- The demultiplexer has several control lines (also known as select lines). These lines determine to which output the input data should be sent. The number of control lines determines the number of output lines.

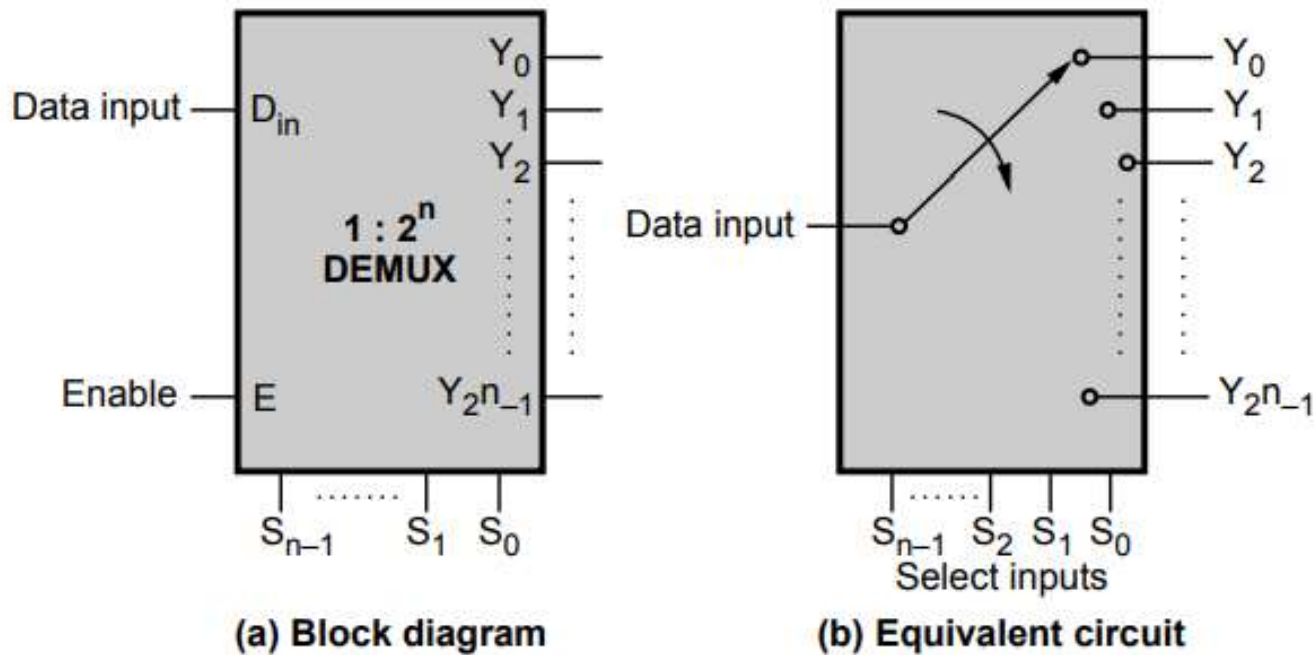
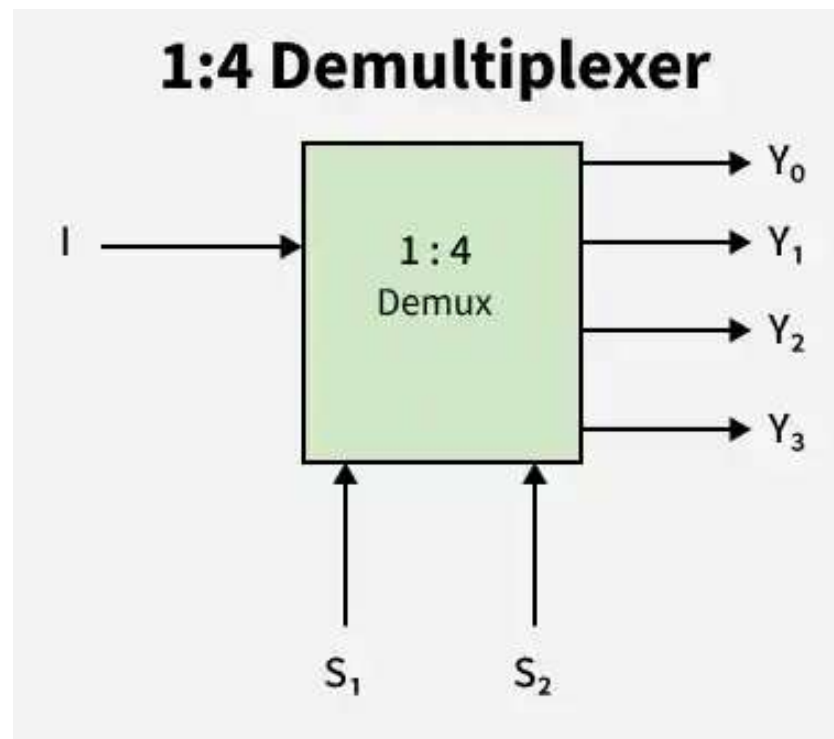
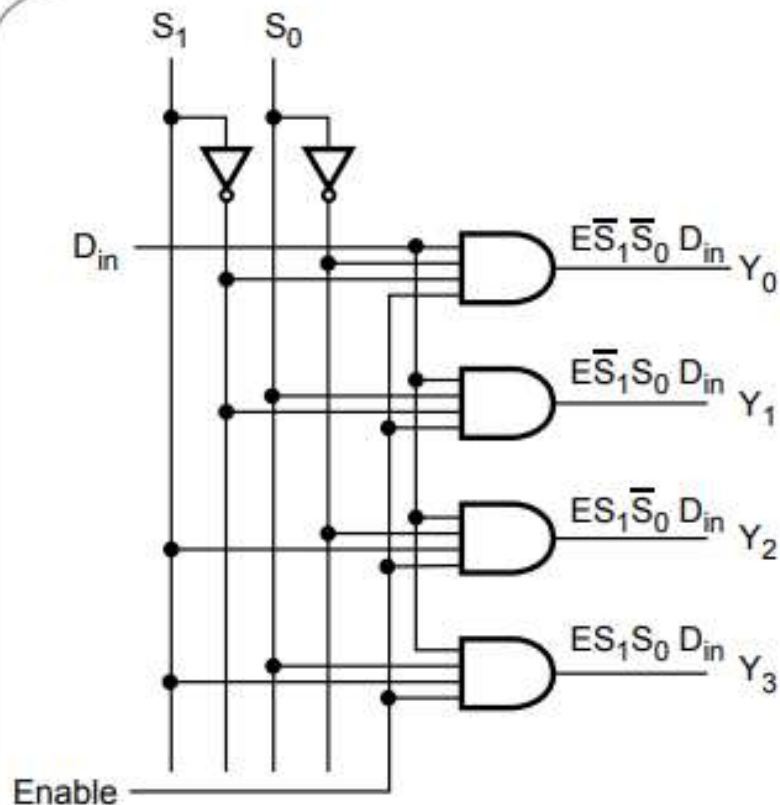


Fig. 3.18.1

Truth Table Of A 1X4 DEMUX

A 1x4 DEMUX has only one input which is denoted as I. There are two selection lines i.e. S1 and S0. At last, the DEMUX has output lines including Y3, Y2, Y1 & Y0. Here is the 1x4 DEMUX with diagram as mentioned below.





(a) Logic diagram

Enable (E)	S_1	S_0	D_{in}	Y_0	Y_1	Y_2	Y_3
0	X	X	X	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	0	0	0	0
1	0	1	1	0	1	0	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	0
1	1	1	0	0	0	0	0
1	1	1	1	0	0	0	1

D_{in} is routed to Y_0
 $Y_0 = E \bar{S}_1 \bar{S}_0 D_{in}$

D_{in} is routed to Y_1
 $Y_1 = E \bar{S}_1 S_0 D_{in}$

D_{in} is routed to Y_2
 $Y_2 = E S_1 \bar{S}_0 D_{in}$

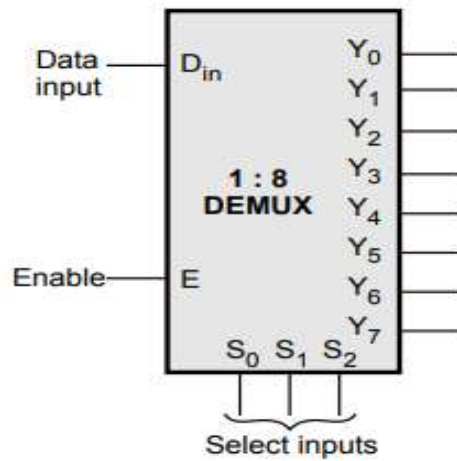
D_{in} is routed to Y_3
 $Y_3 = E S_1 S_0 D_{in}$

(b) Function table for 1 : 4 demultiplexer

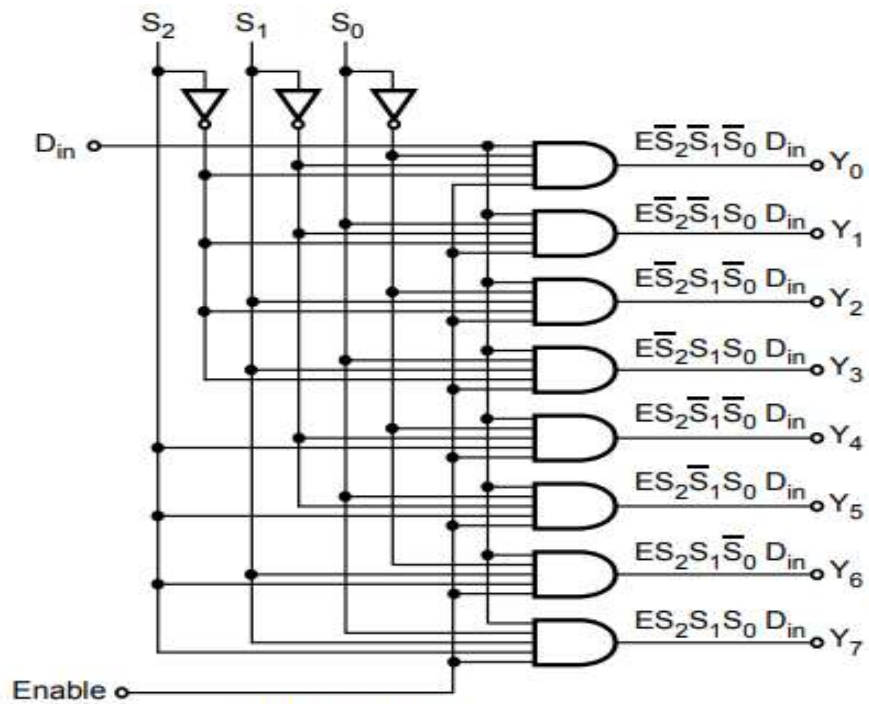
Fig. 3.18.2

Truth Table Of A 1x8 De-Multiplexer

The 1x8 DEMUX was designed by using two DEMUX. They are the two 1x4 DEMUX and one 1x2 DEMUX. The 1x8 DEMUX contains two input lines with four outputs. Let us see the block diagram of the 1x8 DEMUX as mentioned below.



(a) Block schematic



(b) Logic diagram

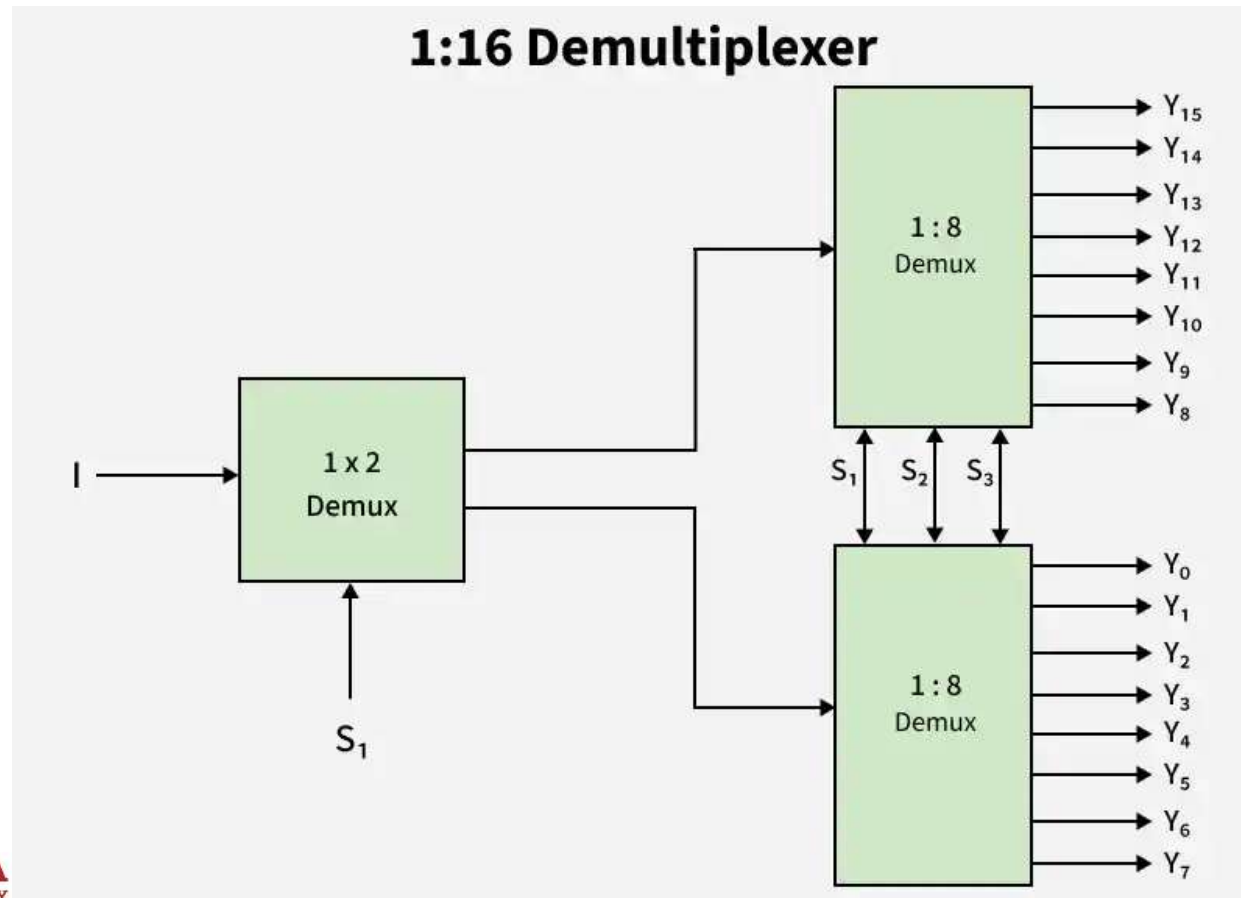
Enable	Select inputs			Outputs								
E	S_2	S_1	S_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7	
0	X	X	X	0	0	0	0	0	0	0	0	
1	0	0	0	D_{in}	0	0	0	0	0	0	0	
1	0	0	1	0	D_{in}	0	0	0	0	0	0	
1	0	1	0	0	0	D_{in}	0	0	0	0	0	
1	0	1	1	0	0	0	D_{in}	0	0	0	0	
1	1	0	0	0	0	0	0	D_{in}	0	0	0	
1	1	0	1	0	0	0	0	0	D_{in}	0	0	
1	1	1	0	0	0	0	0	0	0	D_{in}	0	
1	1	1	1	0	0	0	0	0	0	0	D_{in}	
												$E \bar{S}_2 \bar{S}_1 \bar{S}_0 D_{in}$
												$E \bar{S}_2 \bar{S}_1 S_0 D_{in}$
												$E \bar{S}_2 S_1 \bar{S}_0 D_{in}$
												$E \bar{S}_2 S_1 S_0 D_{in}$
												$E S_2 \bar{S}_1 \bar{S}_0 D_{in}$
												$E S_2 \bar{S}_1 S_0 D_{in}$
												$E S_2 S_1 \bar{S}_0 D_{in}$
												$E S_2 S_1 S_0 D_{in}$

(c) Function table

Fig. 3.18.3 1 : 8 demultiplexer

Truth Table Of A 1x16 De-Multiplexer

The 1x16 DEMUX was designed by using the two 1x8 DEMUX and one 1x2 DEMUX. The 1x16 DEMUX have two input lines. It has eight outputs. Let us see the block diagram of the 1x16 DEMUX as mentioned below.



SELECTION INPUTS				OUTPUTS															
S ₃	S ₂	S ₁	S ₀	Y ₁₅	Y ₁₄	Y ₁₃	Y ₁₂	Y ₁₁	Y ₁₀	Y ₉	Y ₈	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	A	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	A	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	A	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	A	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	A	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	A	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	A	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	A	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	A	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	A	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Ex. 3.18.1 Design 1 : 8 demultiplexer using two 1 : 4 demultiplexers.



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



Step 1 : Connect Din signal to Din input of both the demultiplexers.

Step 2 : Connect select lines B and C to select lines S1 and S0 of the both demultiplexers, respectively.

Step 3 : Connect most significant select line (A) such that when $A = 0$ DEMUX 1 is enabled and when $A = 1$ DEMUX 2 is enabled.

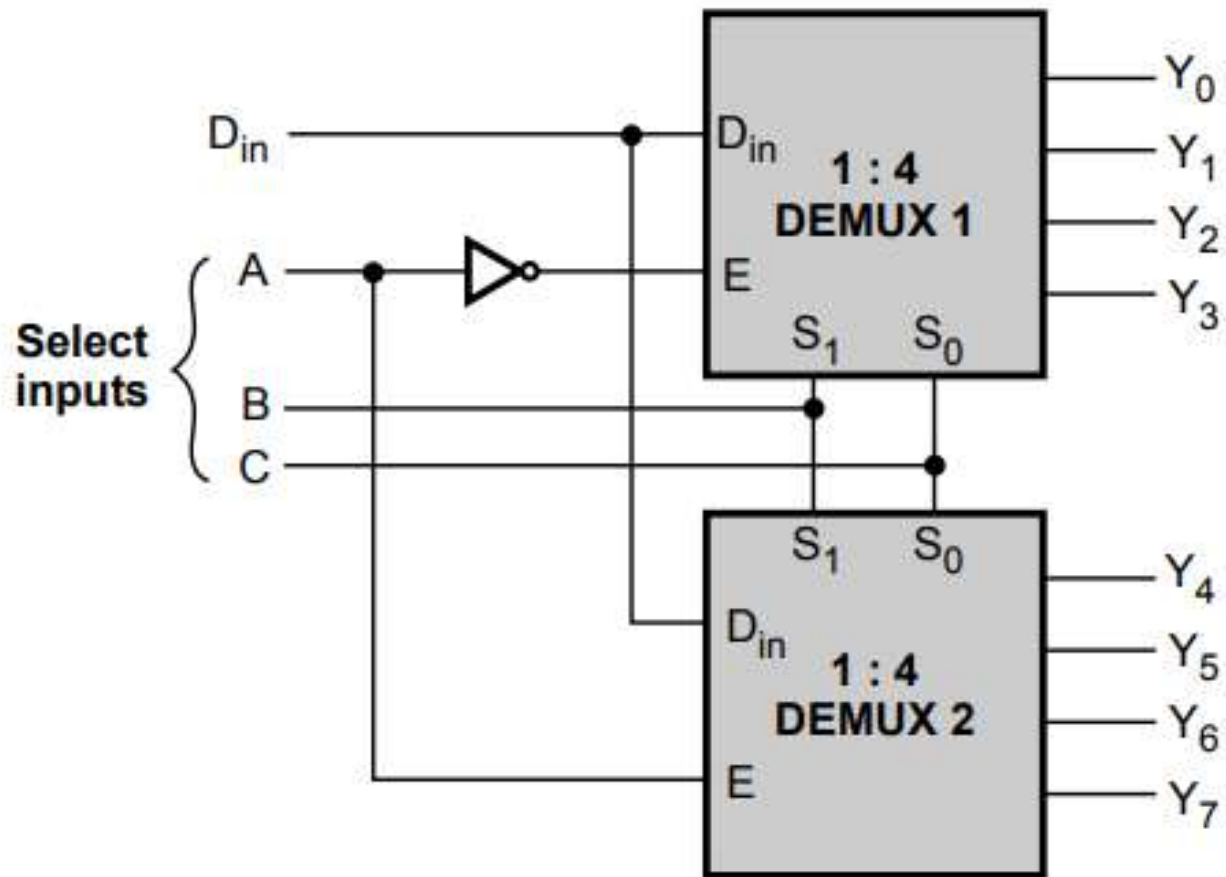


Fig. 3.18.4 Cascading of demultiplexers

Implement 1 : 16 demultiplexer using 1 : 4 demultiplexers.

The 1 : 16 demultiplexer has 16 outputs. To select one of the 16 output, the circuit needs 4 ($2^4 = 16$) select lines. Each 1 : 4 demultiplexer requires 2 select lines.

Step 1 : Connect two least significant select lines (S1, S0) to select lines of four 1:4 demultiplexer.

Step 2 : Connect one more 1:4 demultiplexer such that its four outputs are routed to the data inputs of the four demultiplexers. Connect higher select lines (S3, S2) to the select lines of this demultiplexer.

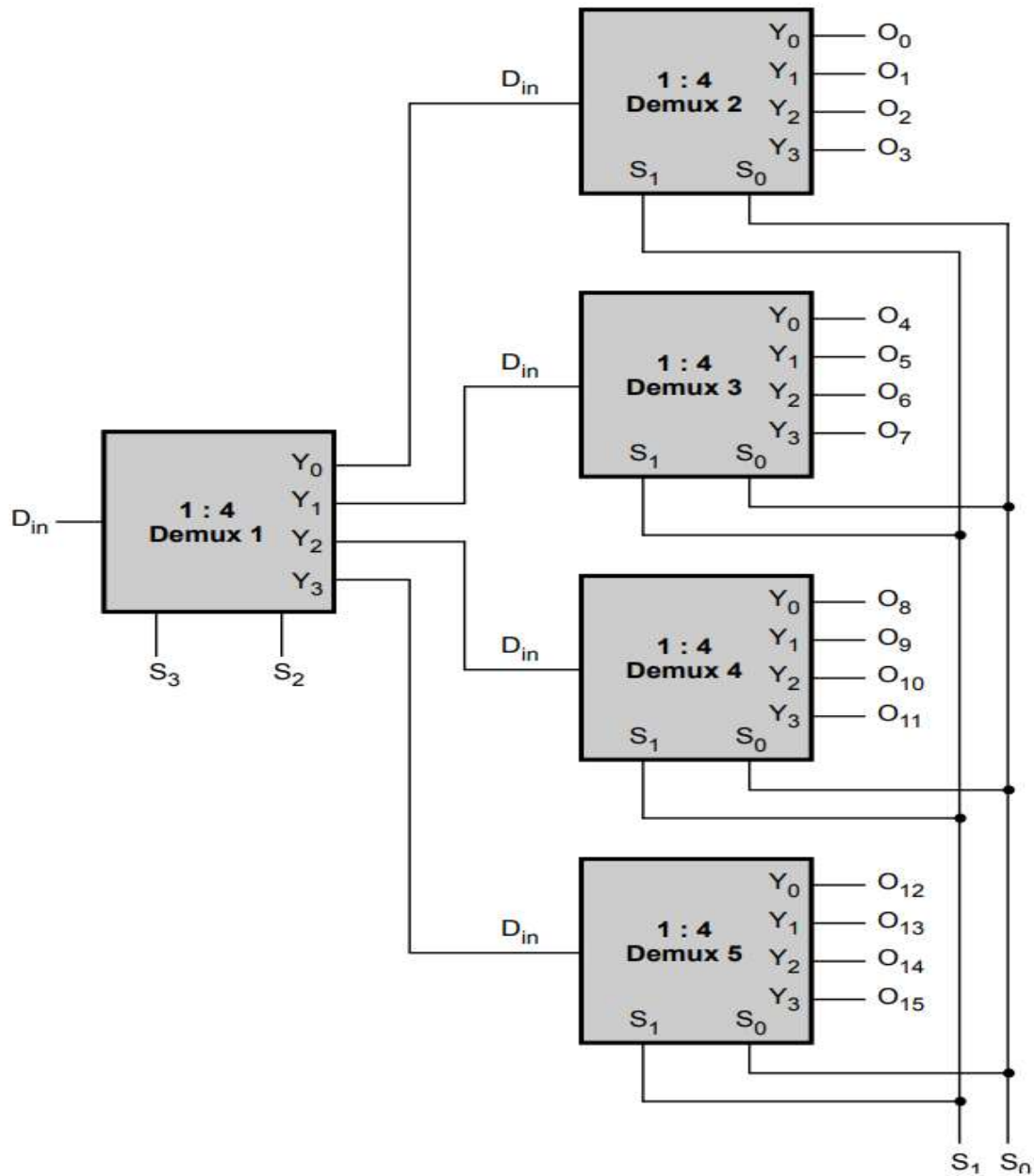


Fig. 3.18.5 1 : 16 Demux using 1 : 4 Demux

Ex. 3.18.6 Implement the following functions using demultiplexer :

$$f_1(A, B, C) = \sum m(0, 3, 7)$$

$$f_2(A, B, C) = \sum m(1, 2, 5)$$

Sol. :

$$f_1(A, B, C) = \sum m(0, 3, 7)$$

$$f_2(A, B, C) = \sum m(1, 2, 5)$$

Implementation using 1 : 8 demultiplexer.

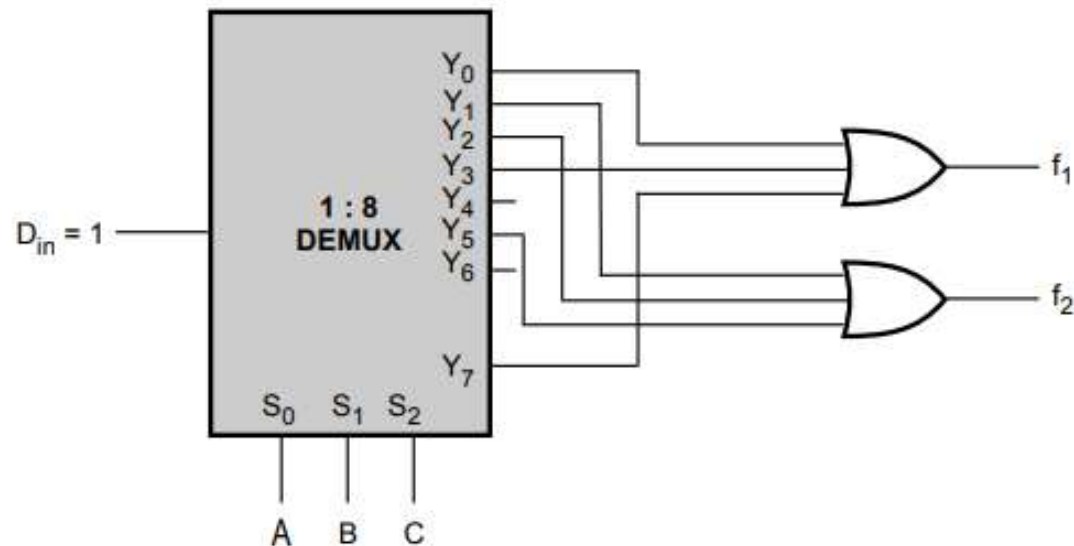


Fig. 3.18.7

Ex. 3.18.5 Implement full subtractor using demultiplexer.

Step 1 : Write the truth table of full subtractor.

Step 2 : Represent output of full-subtractors in minterm form.

A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 3.18.1 Truth table of full subtractor

For full subtractor difference D function can be written as

$$D = f(A, B, C) = \sum m(1, 2, 4, 7)$$

and

Bout function can be written as,

$$Bout = F(A, B, C) = \sum m(1, 2, 3, 7)$$

Step 3: Logically OR the outputs corresponding to minterms.

With D_{in} input 1, demultiplexer gives minterms at the output so by logically ORing required minterms we can implement Boolean functions for full subtractor.

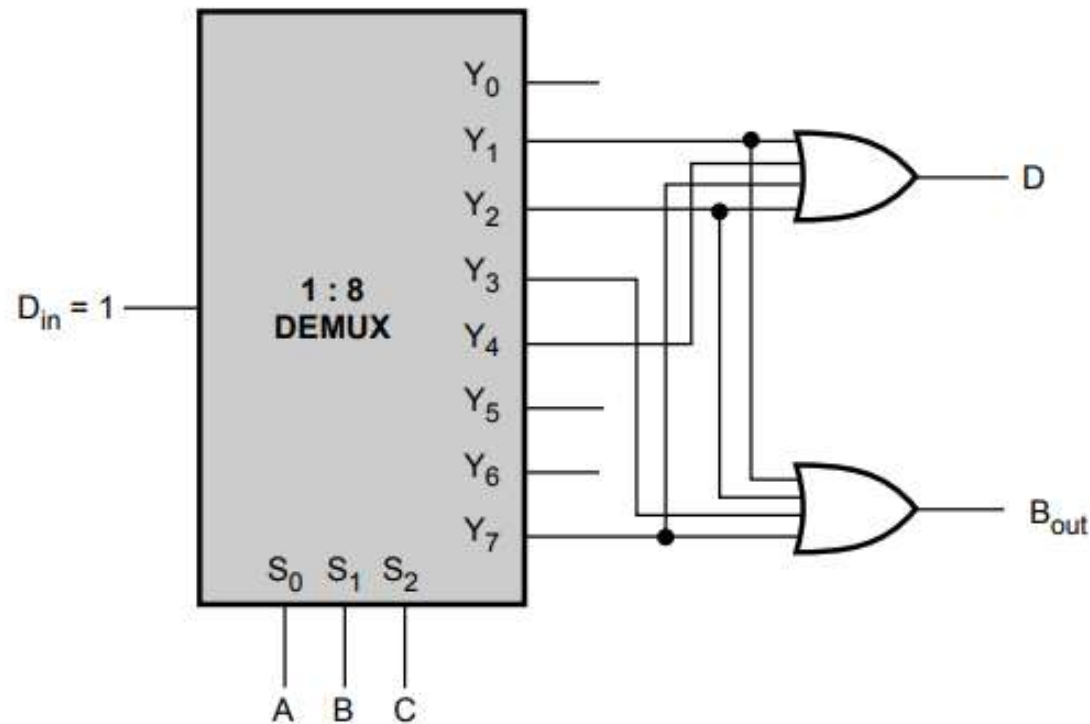


Fig. 3.18.6 Full subtractor using 1 : 8 demultiplexer

Examples for Practice:

Ex. 3.18.7 Implement full adder using demultiplexer.

Ex. 3.18.8 Implement the following functions using demultiplexer

$$f_1(A, B, C) = \sum m(1, 5, 7)$$

$$f_2(A, B, C) = \sum m(3, 6, 7)$$

Advantages of the DEMUX

- The DEMUX increases the efficiency of the particular communication system as it takes data from a specific input source and distributes it to different sources.
- The DEMUX helps to separate the different signals from the mixed data sources. Then it distributes these data to different sources.
- DEMUX can decode the signal outputs of the [multiplexer](#), as the system works in a reverse way of the MUX.

Disadvantages of the DEMUX

- The DEMUX may cause a wastage of bandwidth as it distributes the refined data in different channels. These channels can overlap with each other which leads to the loss of signal.
- The DEMUX may cause problems in the synchronization of signals. The data channels can overlap with each other which leads to the delay in the whole process.

Applications of DEMUX

- DEMUXs are majorly used to design the ALU circuits and parallel data segments.
- The DEMUXs convert the output of the MUX into the actual input. On the receiver end, the DEMUX can be used to verify the original form of the data to carry out the entire communication process.
- DEMUX helps to save the output. The output generally saved to the ALU. The output will be saved in the registers and the various storage units of the system.
- Each DEMUX has a connection with multiple registers which helps to store the processed data into it.
- The counterpart of the DEMUX regulates the data signal at the output stage of the DEMUX operation. These data can be retrieved later to read out parallelly.
- In the audio or video system, the DEMUX distributes them in different channels. In broadcasting, the DEMUX separates the composite signal into an individual one.